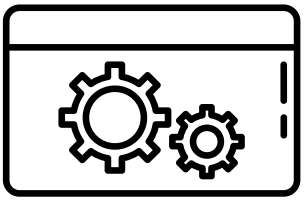
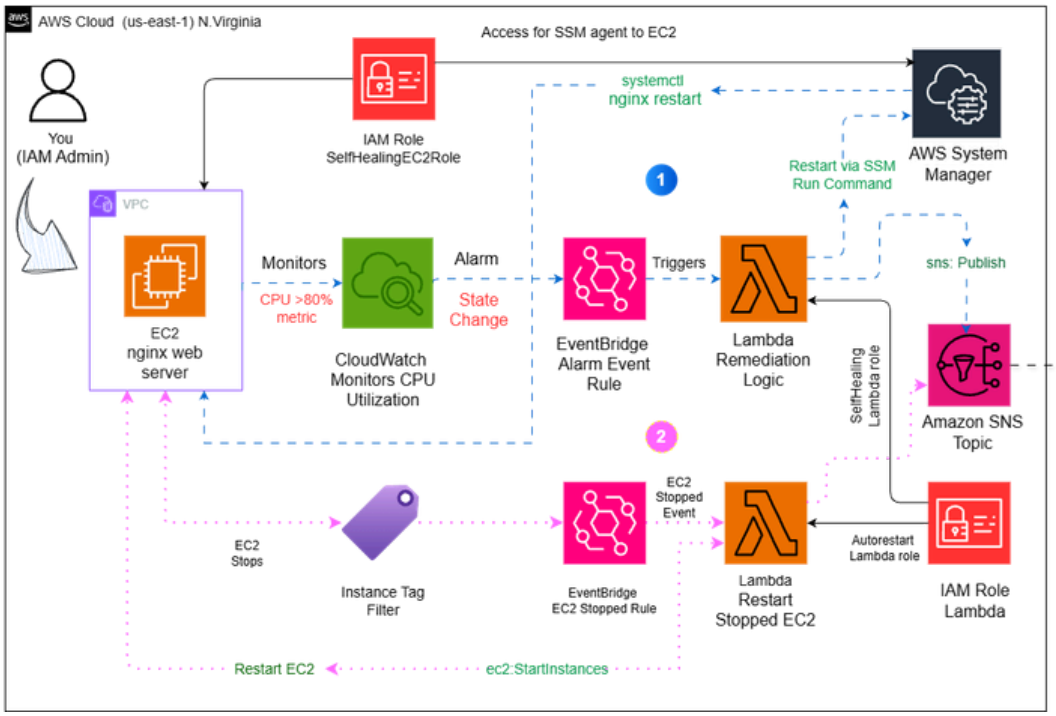


Kareshma
Rajaananthapadmanaban

Event Driven Self Healing DevOps Monitoring System on AWS

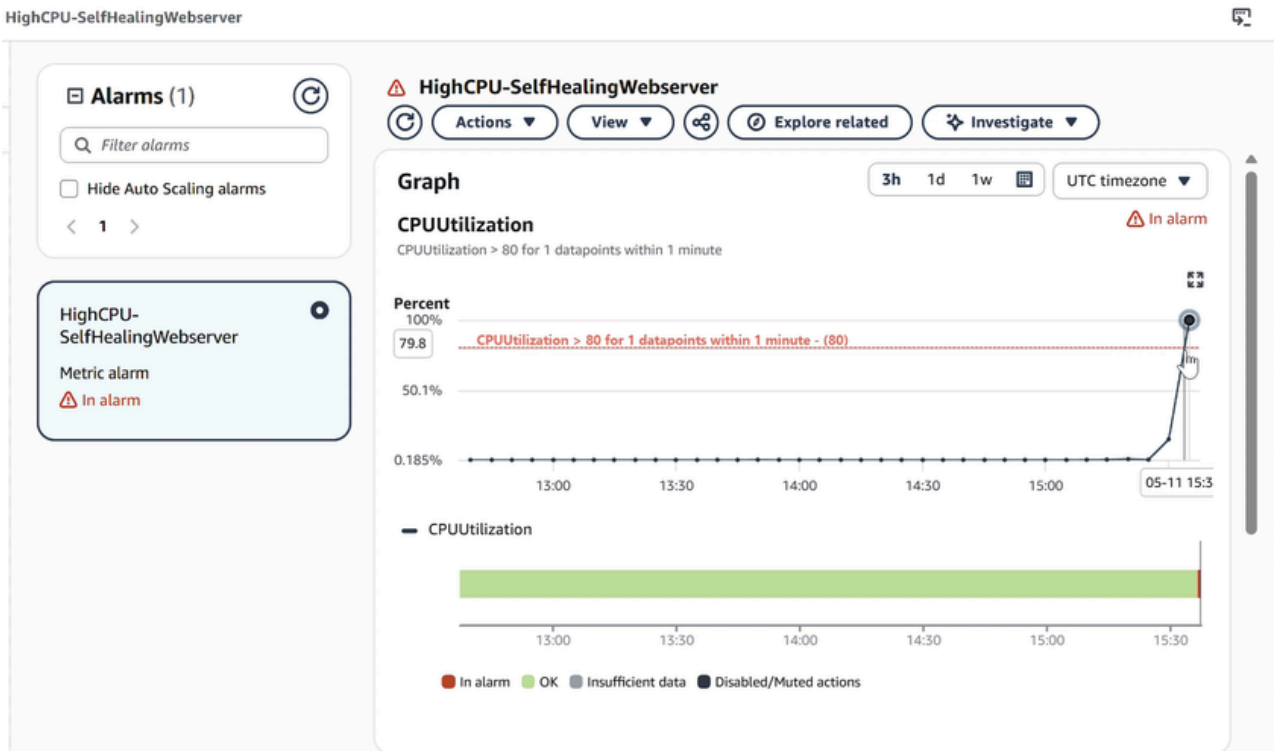
Automate EC2 failure detection and recovery using EventBridge and Lambda.



Building a Self-Healing Infrastructure on AWS

Kareshma Rajaananthapadmanaban

Project Type: Event-Driven DevOps Automation Focus Areas: AWS, DevOps Automation, Observability, Self-Healing System



Introduction

This project demonstrates how to build a self-healing infrastructure using event driven AWS services. The system detects failures in a web server automatically and remediates the issue without manual intervention. The pipeline combines monitoring, event routing, serverless remediation, and automated notifications.

Problem Statement

In production environments, service failures such as crashed web servers or accidentally stopped instances can cause downtime. Manual troubleshooting increases response time and operational overhead. This project addresses that problem by implementing an automated remediation pipeline that detects anomalies and fixes them automatically

Solution Architecture

The architecture uses monitoring, event routing, and serverless automation to detect and remediate failures. When a service failure or resource state change occurs, monitoring tools trigger events that invoke Lambda functions responsible for corrective actions.

Architecture Overview

Architecture Workflow:

1. The nginx web server runs on an EC2 instance.
2. CloudWatch monitors CPU utilization and service health.
3. When CPU usage exceeds the defined threshold or service failure occurs, a CloudWatch alarm is triggered.
4. EventBridge captures the alarm state change event.
5. EventBridge invokes the remediation Lambda function.
6. Lambda restarts nginx through Systems Manager Run Command.
7. SNS sends an email notification describing the remediation action to the user.
8. EventBridge detects EC2 stopped event and Lambda checks for AutoRestart=true tag.
9. Instance is restarted automatically and SNS notification is sent.

AWS Services Used

Service	Purpose
EC2	Hosts the nginx web server used for testing failures
CloudWatch	Monitors CPU metrics and triggers alarms
EventBridge	Routes events from alarms and EC2 state changes
Lambda	Executes automated remediation logic
Systems Manager (SSM)	Runs commands on EC2 instances
SNS	Sends email notifications
IAM	Controls permissions for EC2 and Lambda roles

Infrastructure Components

- EC2 Instance: SelfHealing-Webserver (t3.micro)

IAM Roles: -

- SelfHealingEC2Role
- SelfHealingLambdaRole
- AutoRestartLambdaRole

Monitoring: -

- CloudWatch Alarm: HighCPU-SelfHealingWebserver

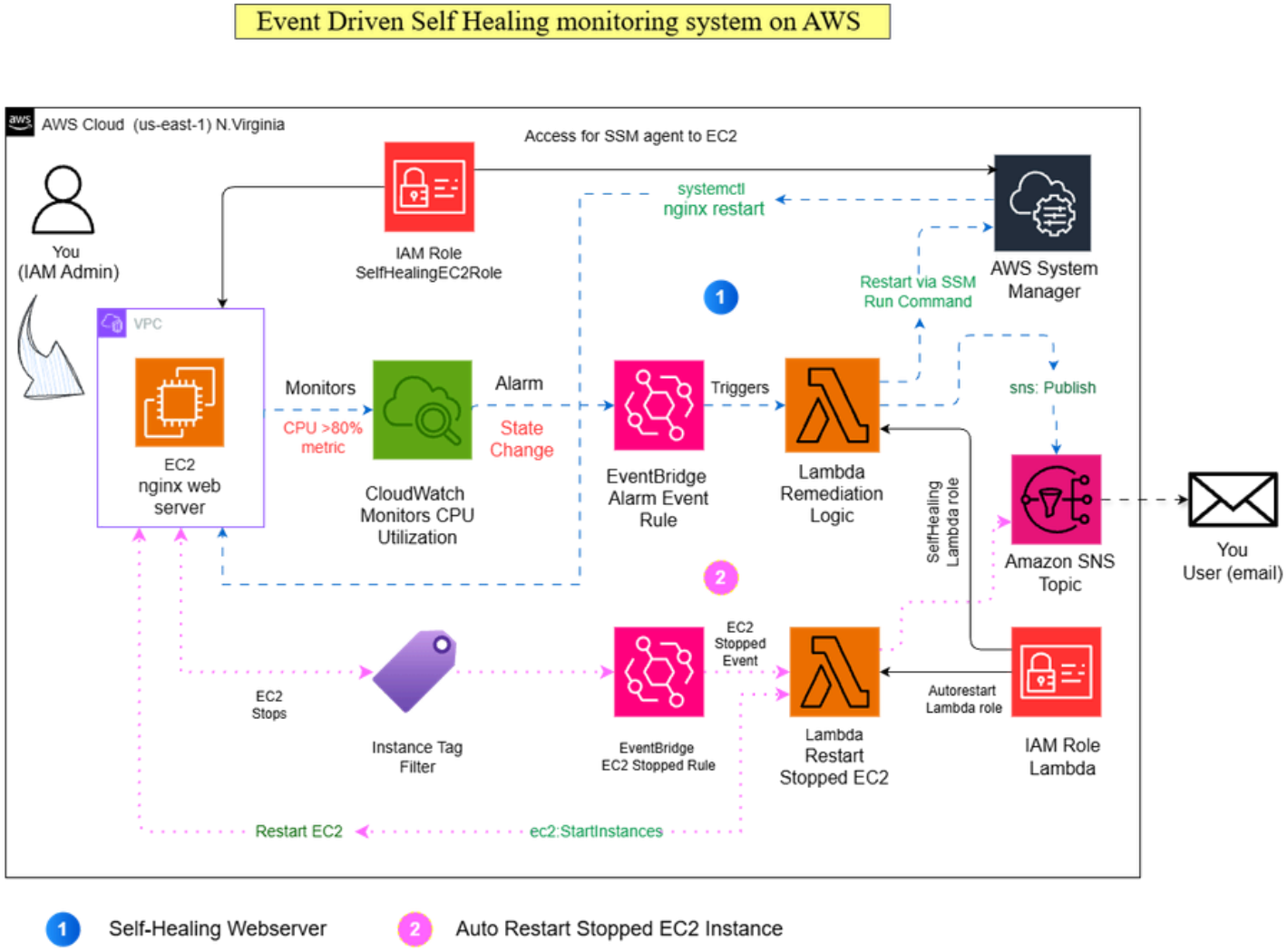
Automation: -

- EventBridge Rule: HighCPU-Remediation-Rule
- EventBridge Rule: EC2-AutoRestart-Rule
- Lambda Function: SelfHealingRemediation
- Lambda Function: AutoRestartInstance

Notifications: -

- SNS Topic: SelfHealingAlerts

Architecture Diagram



Project overview and goals

In this project, I'm building an event-driven self-healing system on AWS that detects nginx web server failures running on an EC2 instance and automatically restarts the service using AWS Lambda and Systems Manager. The system uses CloudWatch alarms and EventBridge rules to monitor failures and trigger remediation, while Amazon SNS sends notifications confirming that the issue was detected and automatically resolved.

Reflections and Key Takeaways

Tools and concepts mastered

The key tools I used include Amazon EC2, Amazon CloudWatch, Amazon EventBridge, AWS Lambda, Amazon SNS, AWS Systems Manager, and AWS Identity and Access Management. Key concepts I learned include building an event-driven automation workflow, monitoring infrastructure using CloudWatch alarms, triggering remediation with Lambda, and designing a self-healing system that automatically detects failures, restarts services, and sends alerts without manual intervention.

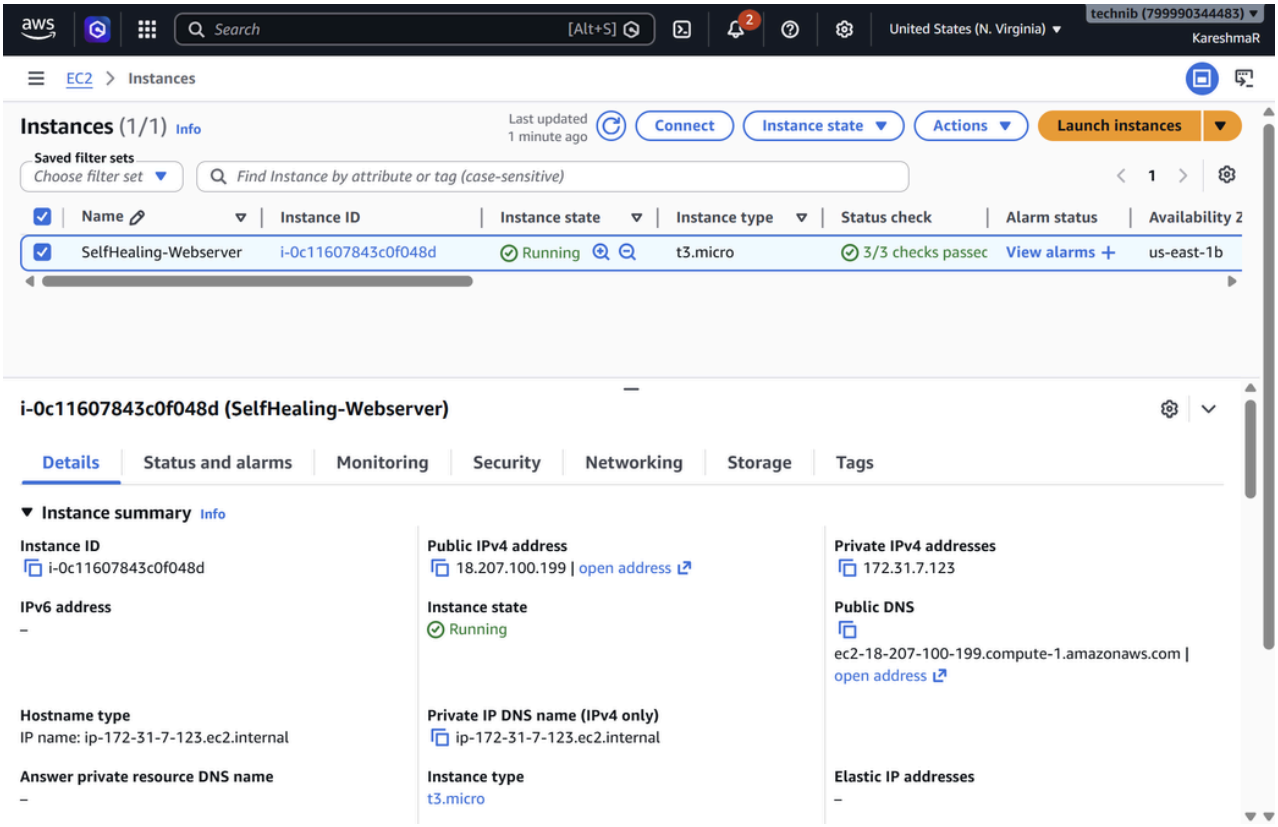
Time and challenges

This project took me approximately 1 hour and 30 minutes to complete. The most challenging part was troubleshooting an invalid instance ID error, which happened because my Lambda function was created in a different AWS region while the EC2 instance and monitoring resources were running in another region. Once I aligned all resources in the same region, the pipeline worked correctly.

Deploying the nginx Web Server on EC2

Setting up EC2 with IAM and nginx

In this step, I'm setting up an EC2 instance running an nginx web server and configuring an IAM role that allows AWS Systems Manager to securely access and manage the instance. I attach the role during launch and install nginx using a user data startup script so the web server starts automatically. This creates the base environment for the project, allowing me to monitor the server and later automate failure detection and recovery through the self-healing pipeline.



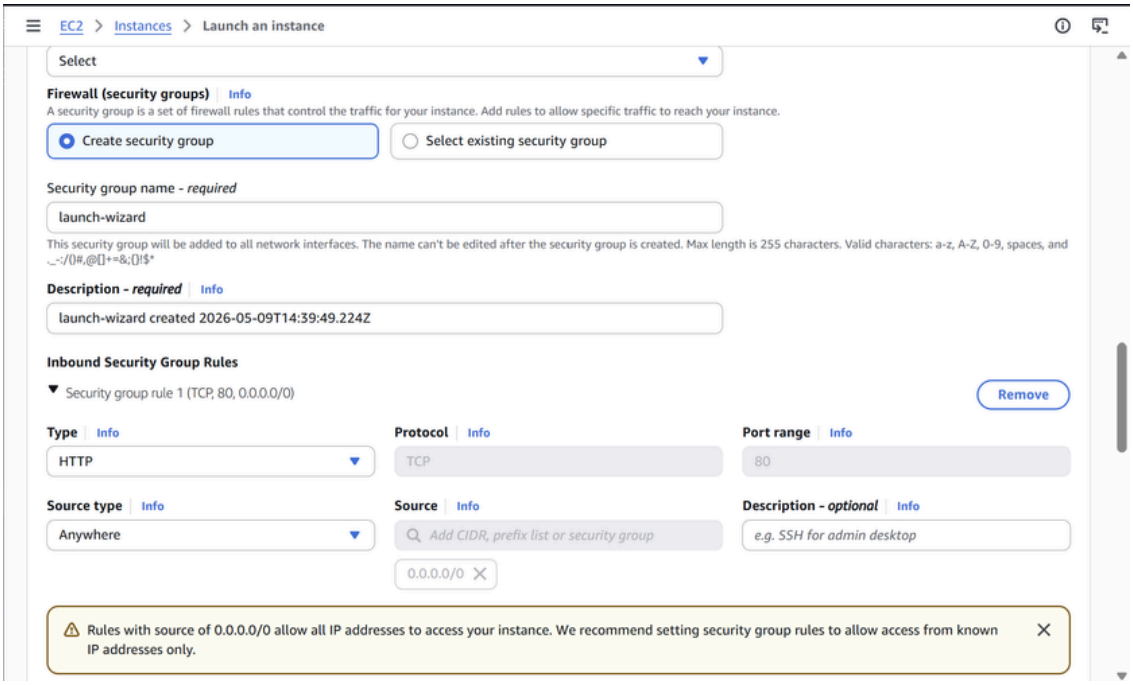
Launch EC2 instance with startup script under user data

Deploying the nginx Web Server on EC2

Setting up EC2 with IAM and nginx

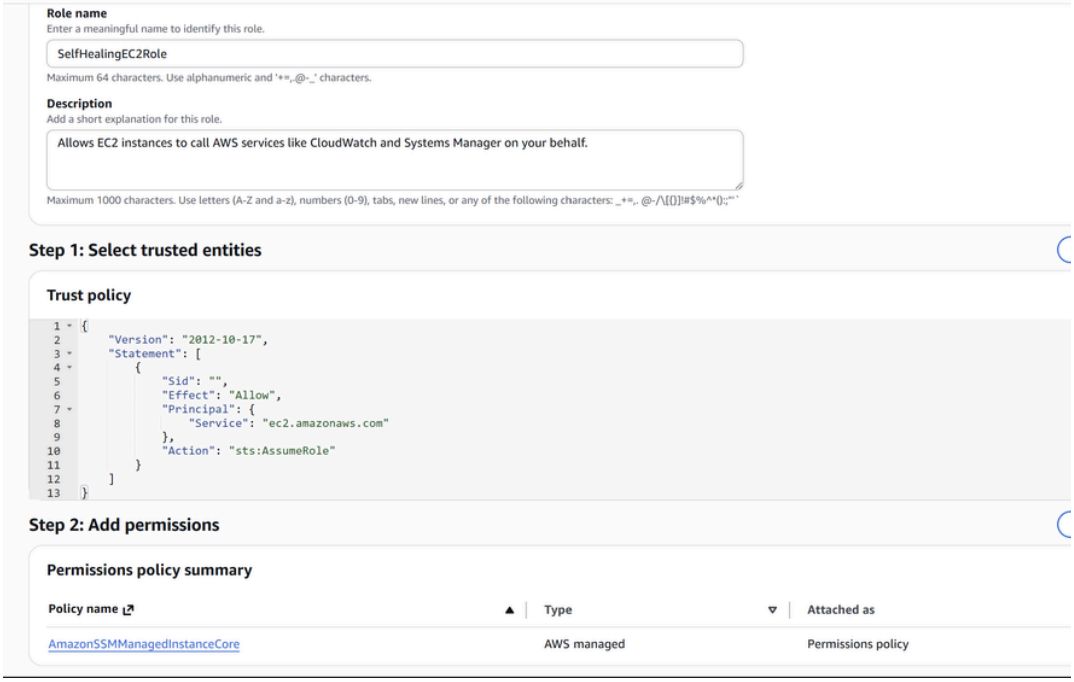
In this step, I'm setting up an EC2 instance running an nginx web server and co

```
$ Startup-script.sh
1   #This script runs automatically when your EC2 instance boots for the first time.
2
3   ##!/bin/bash tells the instance to run this as a shell script.
4   #!/bin/bash
5
6   #sudo dnf install -y nginx installs the nginx web server using the Amazon Linux 2023 package manager.
7
8   sudo dnf install -y nginx
9
10  #sudo systemctl start nginx starts the web server immediately.
11
12  sudo systemctl start nginx
13
14  #sudo systemctl enable nginx ensures nginx restarts automatically if the instance reboots.
15
16  sudo systemctl enable nginx
```

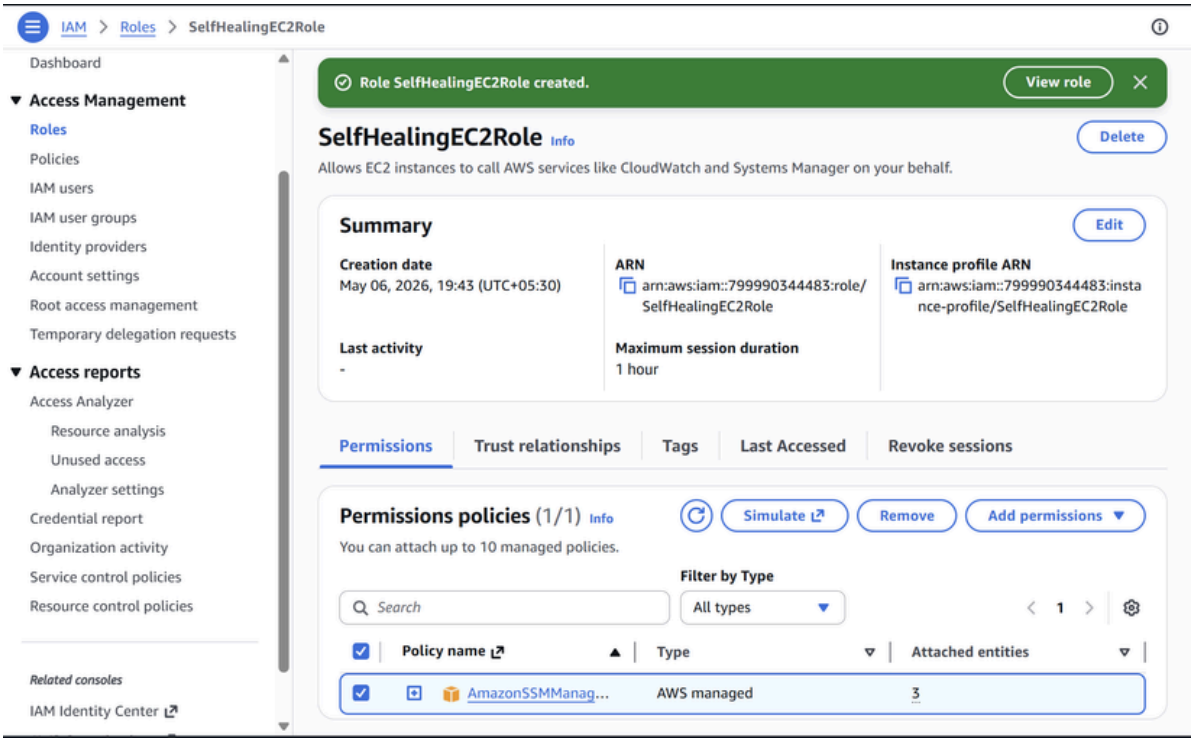


Network set up - Security group

Setting up EC2 with IAM and nginx



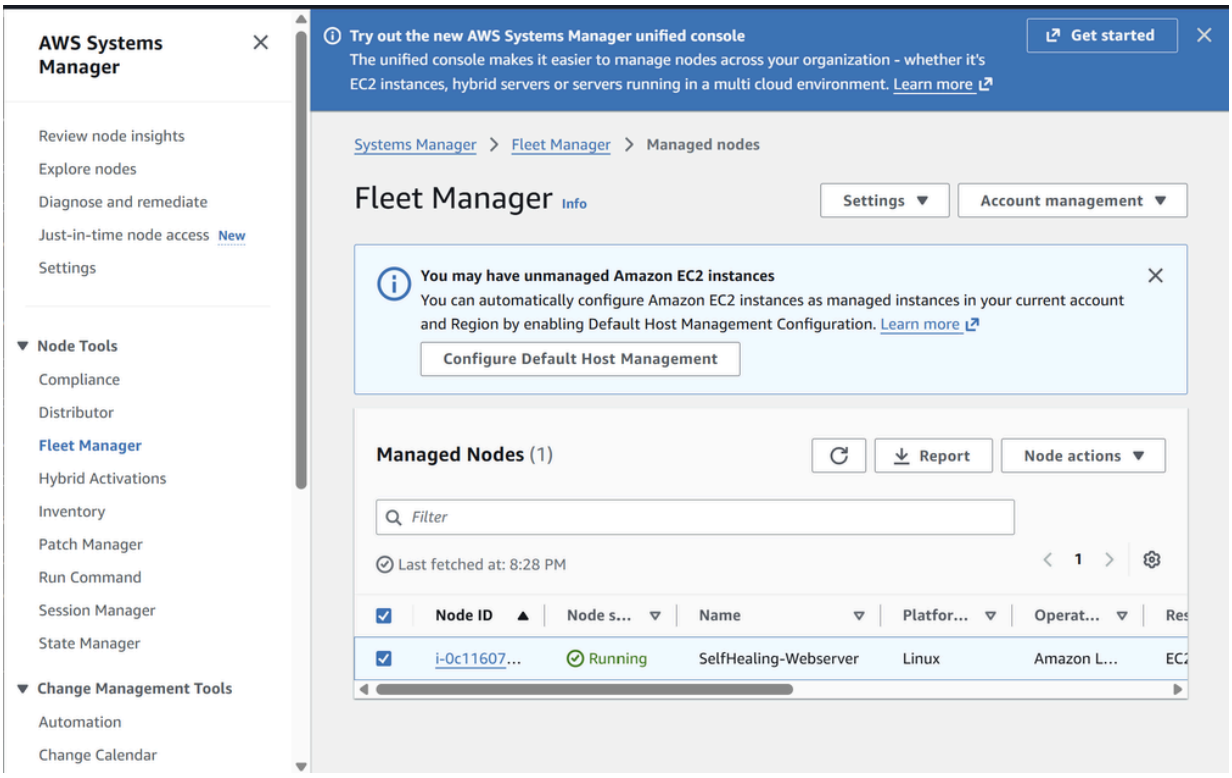
IAM Role setup



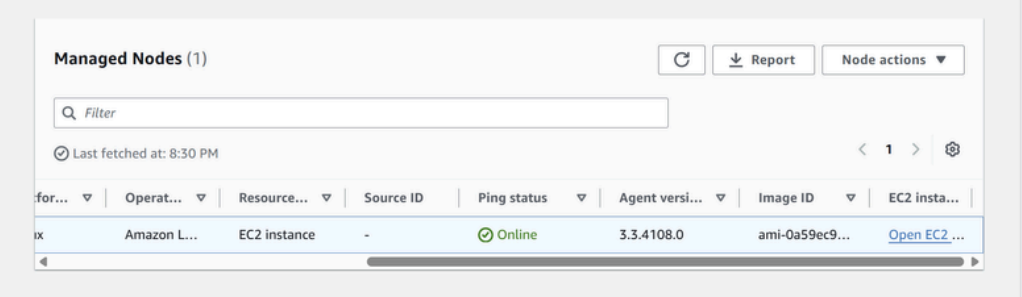
IAM Role

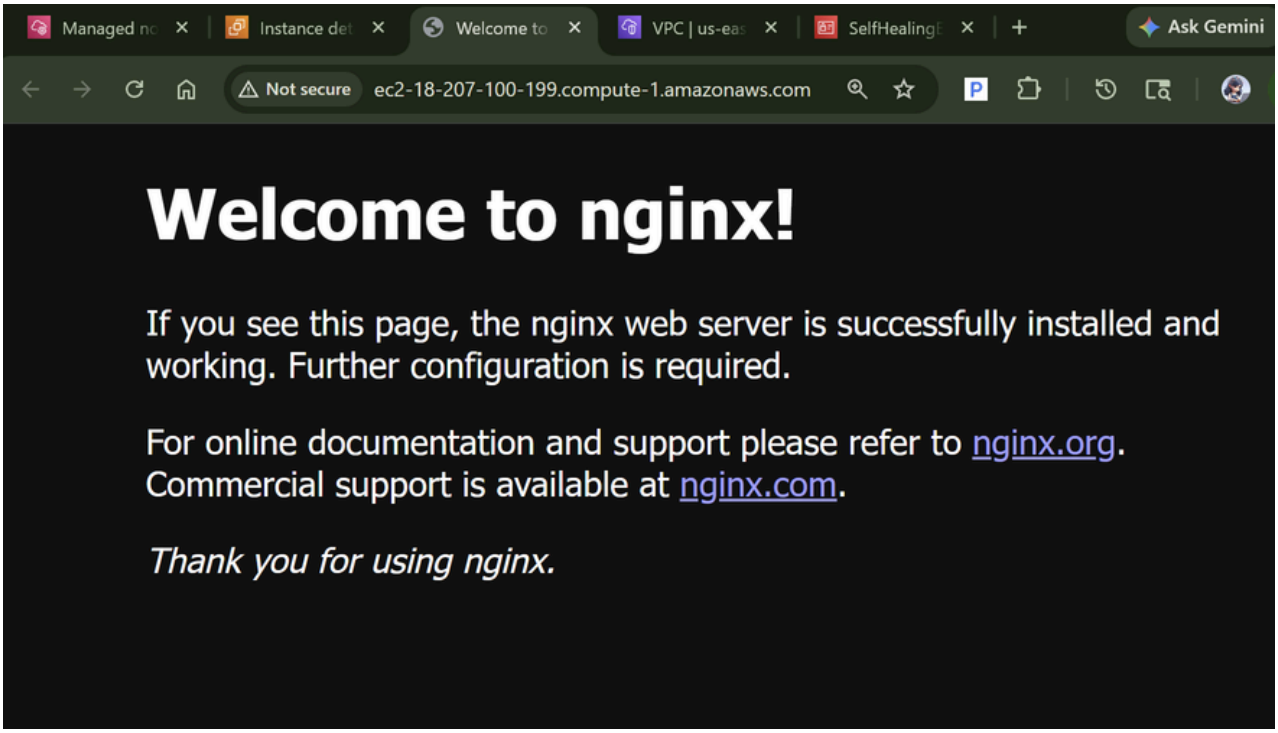
IAM role permissions for Systems Manager

The IAM role allows the instance to securely communicate with AWS Systems Manager so it can receive and execute remote management commands. In this project, it enables the pre-installed SSM Agent on the EC2 instance to register with Systems Manager and accept commands without requiring SSH access. This permission allows services like Lambda to remotely run commands on the instance, such as restarting the nginx web server during automated failure recovery.



SSM Fleet Manager



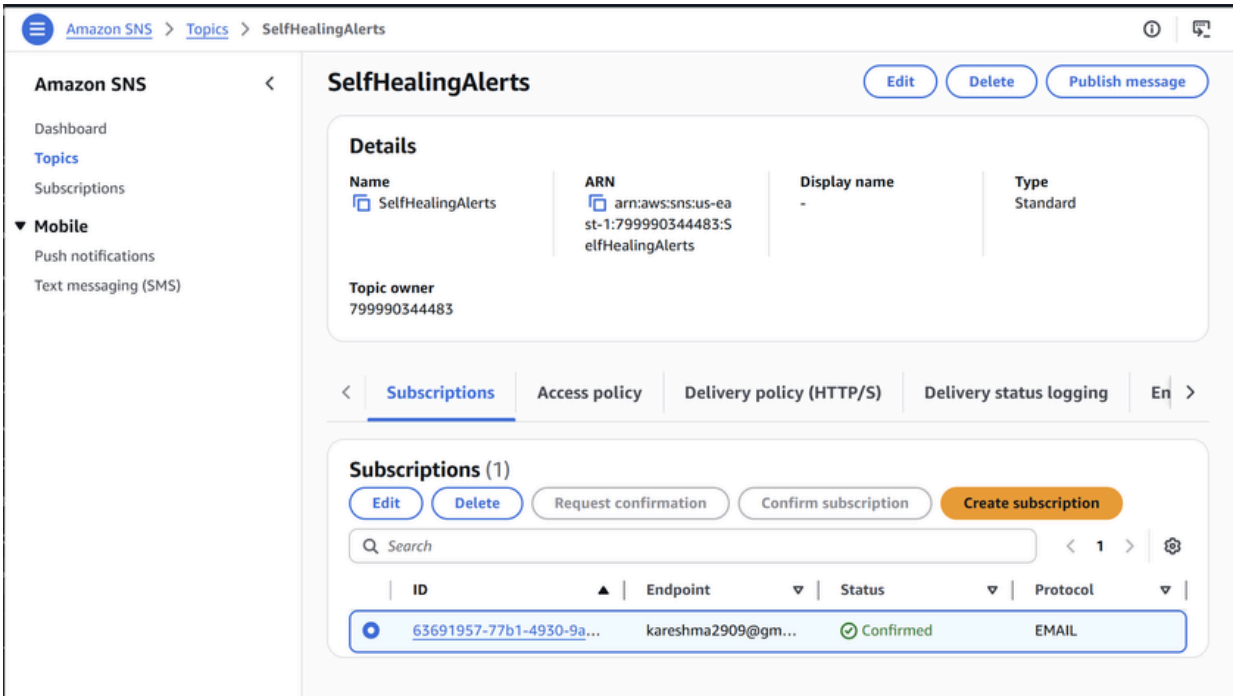


nginx web page

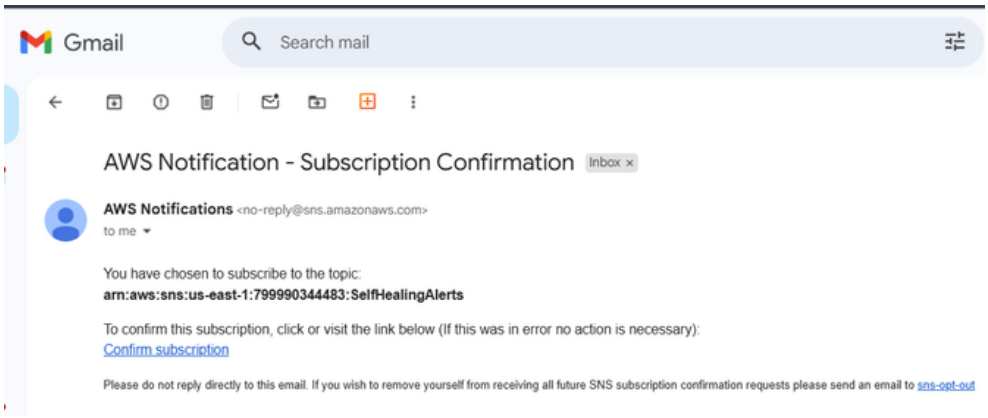
Configuring Alerts with SNS and CloudWatch

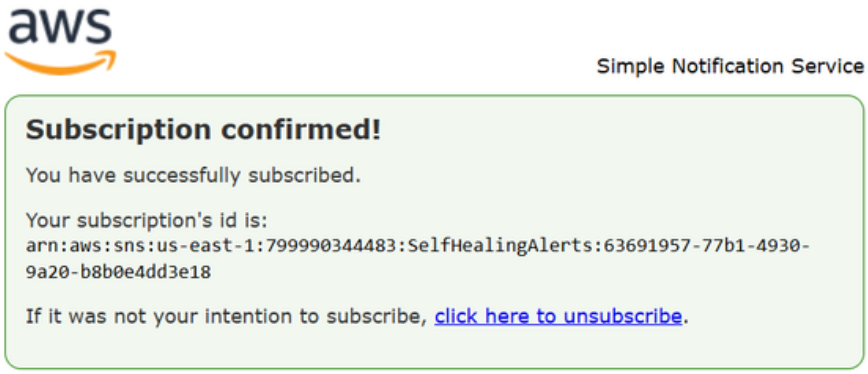
Setting up the notification and monitoring layer

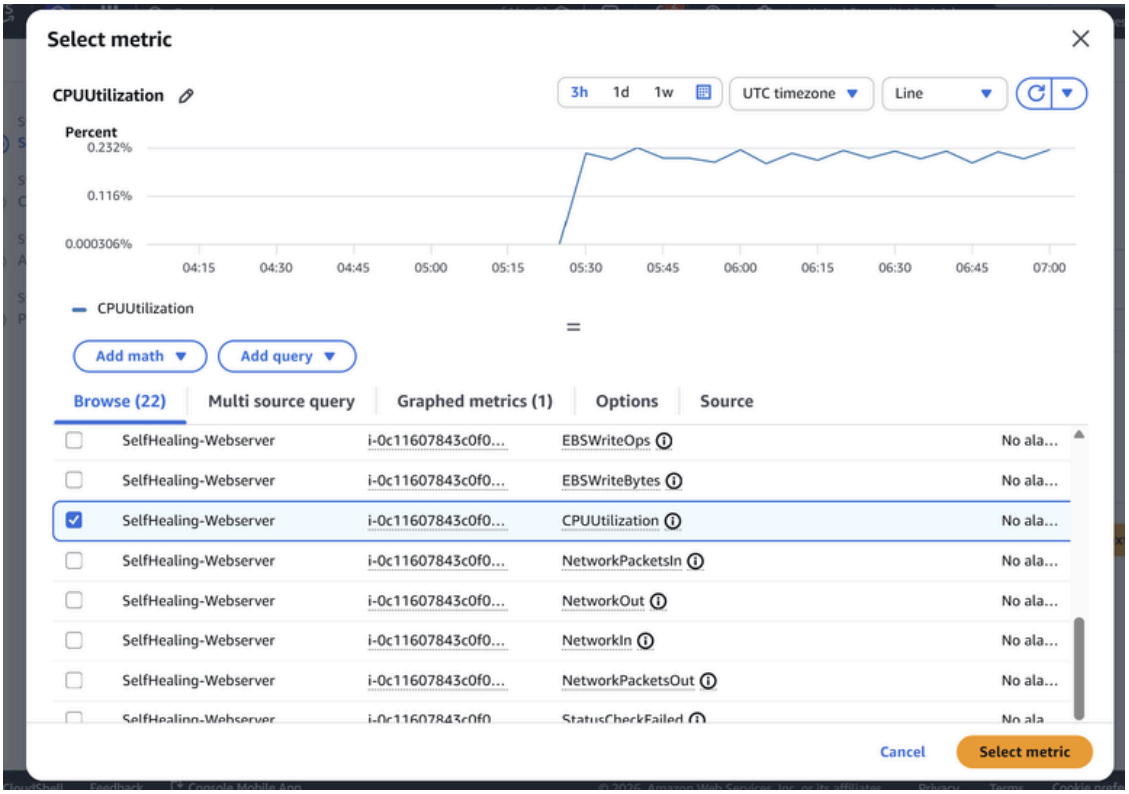
In this step, I'm setting up an alerting and monitoring layer using Amazon SNS and CloudWatch so that I can automatically detect abnormal activity on my EC2 instance and receive notifications when a threshold is triggered. I create an SNS topic with an email subscription for alerts and configure a CloudWatch alarm to monitor CPU utilization. When CPU usage exceeds the defined limit, the alarm sends a notification through SNS, enabling the monitoring pipeline to react to potential failures.



SNS Topic with subscription



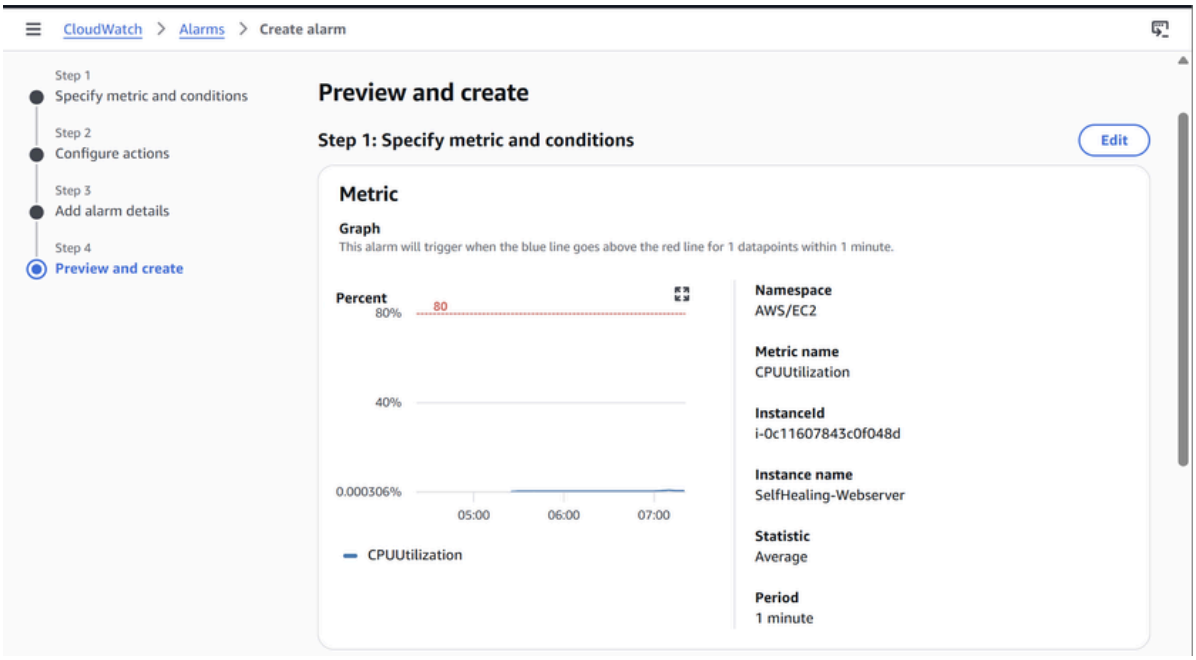




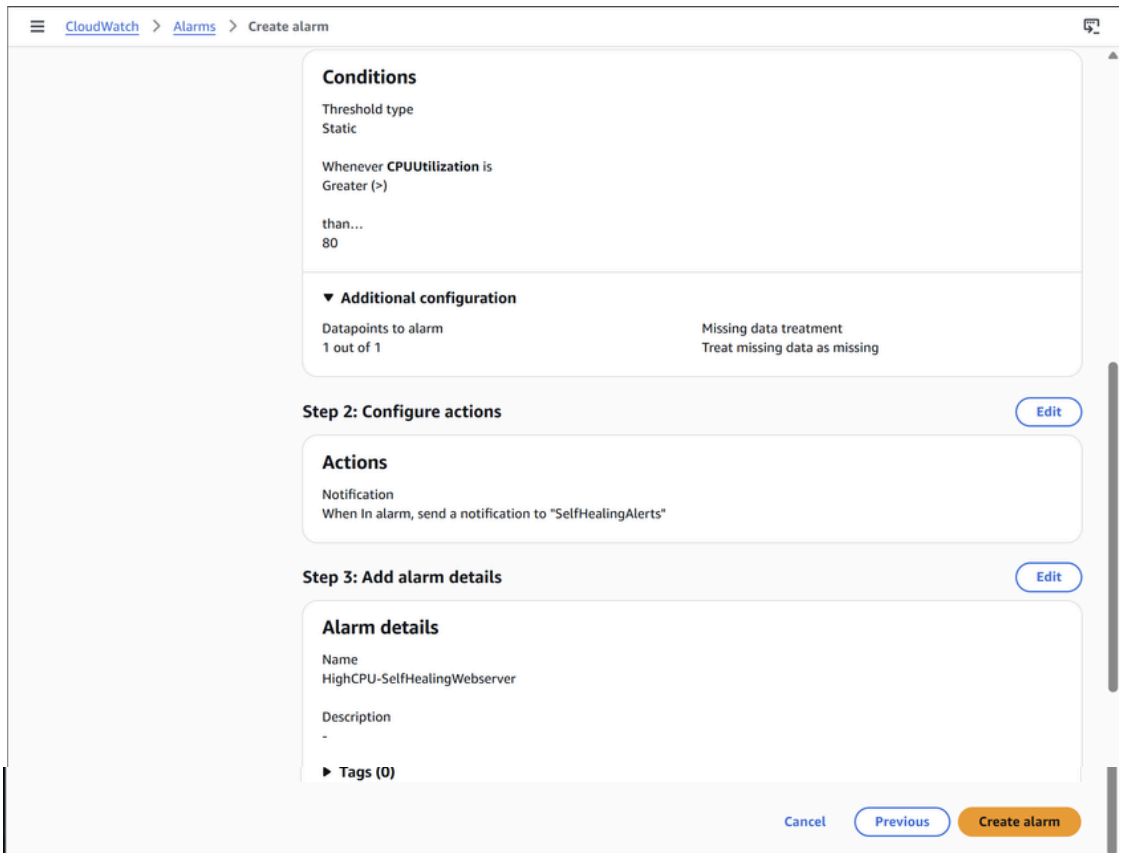
Metric of EC2 CPU Utiliztion

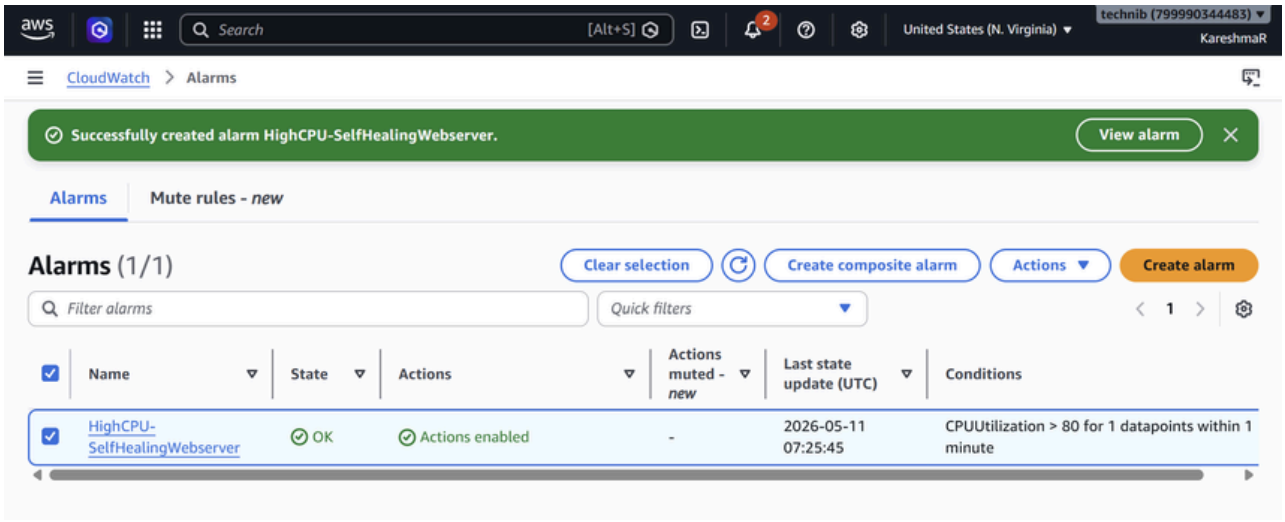
Alarm trigger conditions and actions

The alarm triggers when the EC2 instance's CPU utilization exceeds 80% during the 1-minute evaluation period. In this project, I configured the alarm to evaluate one datapoint, so a single reading above the threshold immediately changes the alarm state to In Alarm. When this condition is met, CloudWatch publishes a notification to the SelfHealingAlerts SNS topic, which then sends an email alert to my subscribed address to notify me that the monitoring system detected abnormal activity.



Configure alarm for CloudWatch





CloudWatch alarm list

Writing the Lambda Remediation Function

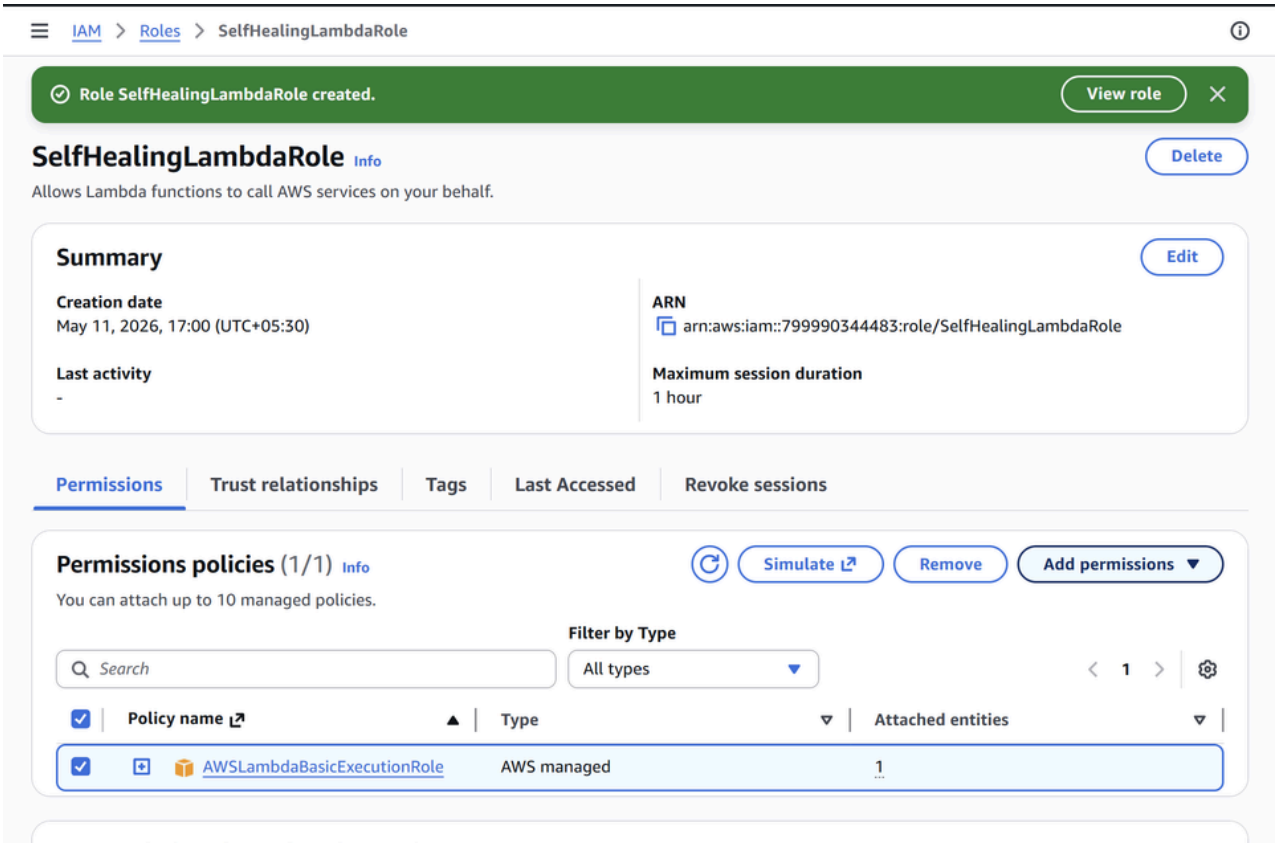
Building automated remediation logic in Python

In this step, I'm building the automated remediation component using AWS Lambda and IAM so that the system can respond to failures without manual intervention. I create a Lambda function with the required permissions to interact with Systems Manager and SNS. The function receives alarm events, identifies the affected EC2 instance, and remotely executes a command to restart the nginx service. After performing the remediation, it also sends a notification confirming that the system automatically detected and resolved the issue.

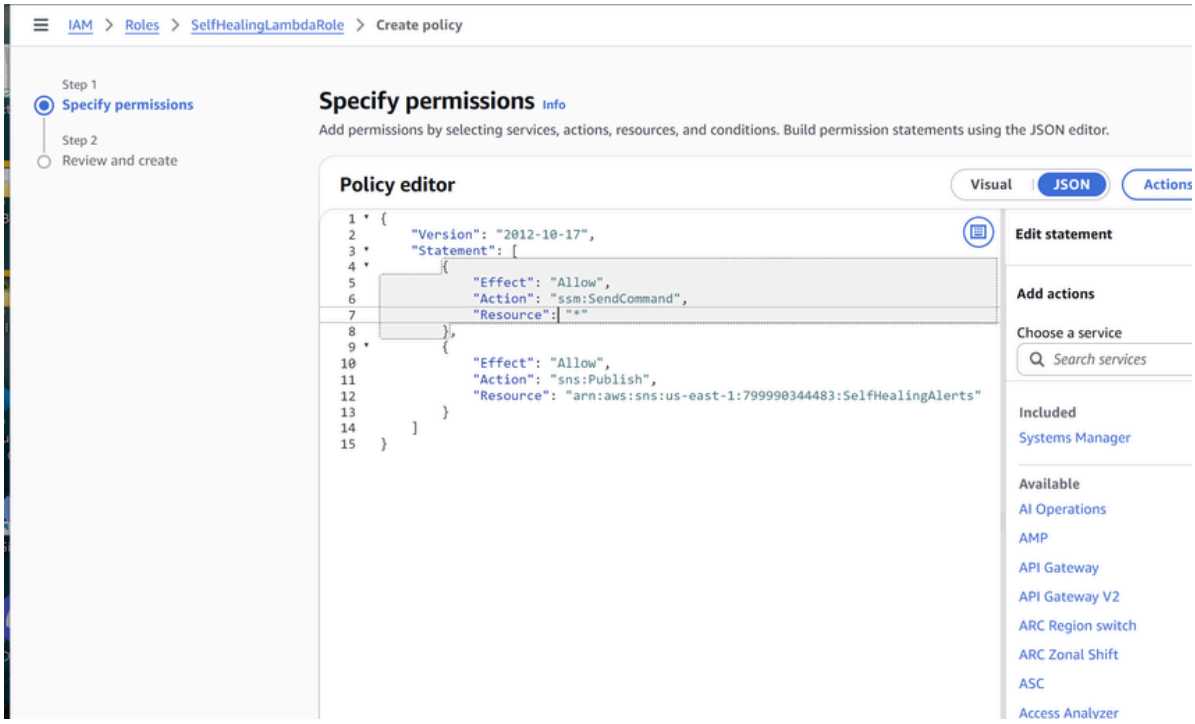
Lambda Remediation Logic

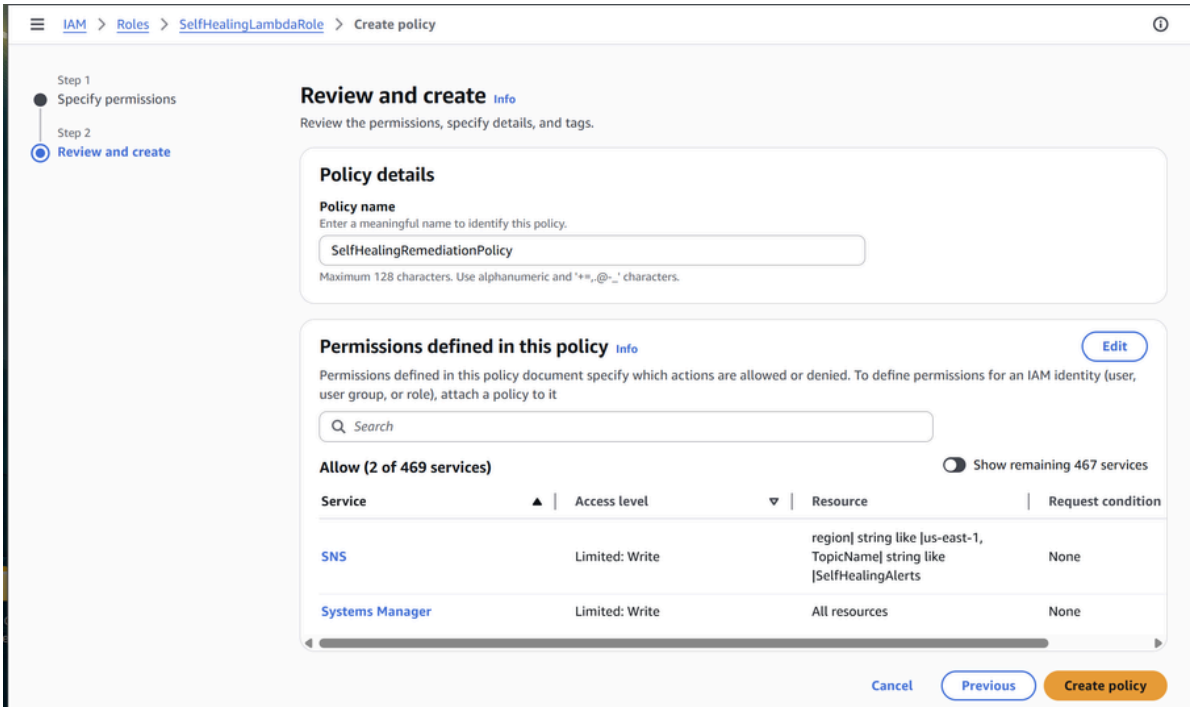
The Lambda function performs the following tasks:
- Receives CloudWatch alarm events
- Extracts instance ID from event payload
- Sends SSM Run Command to restart nginx
- Publishes notification to SNS

Example command executed via SSM: sudo systemctl restart nginx

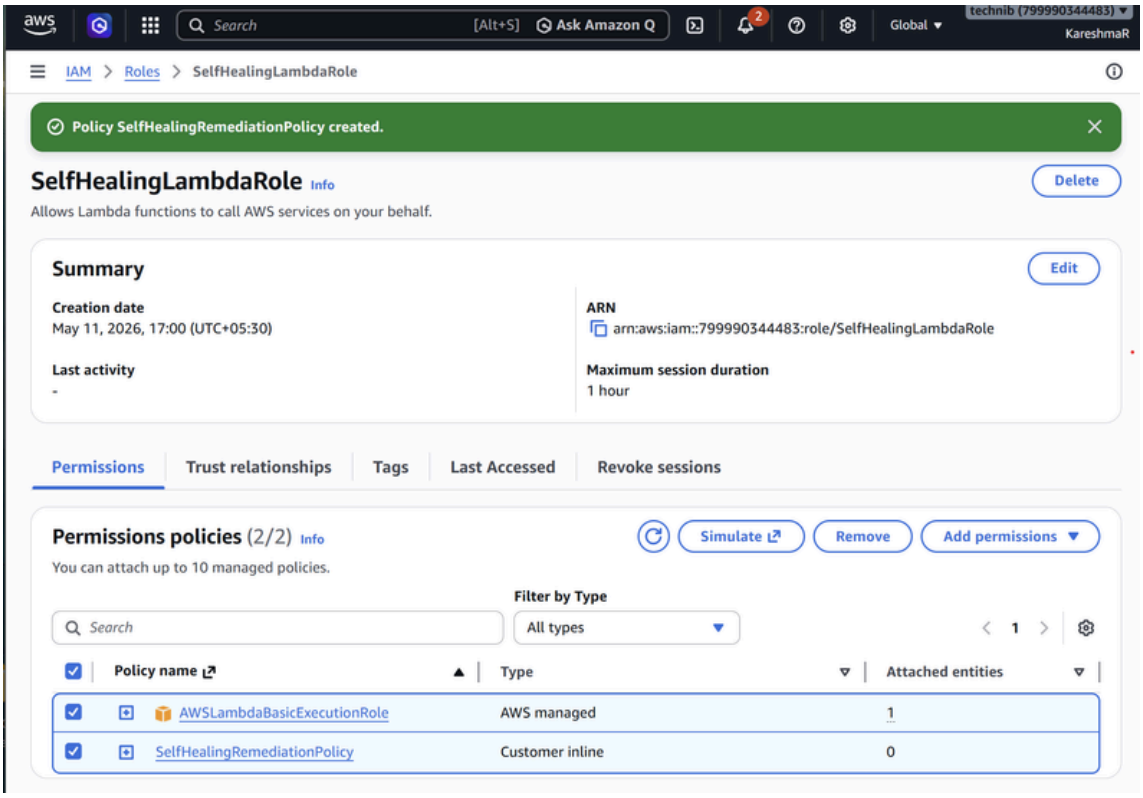


Lambda execution role and inline policy

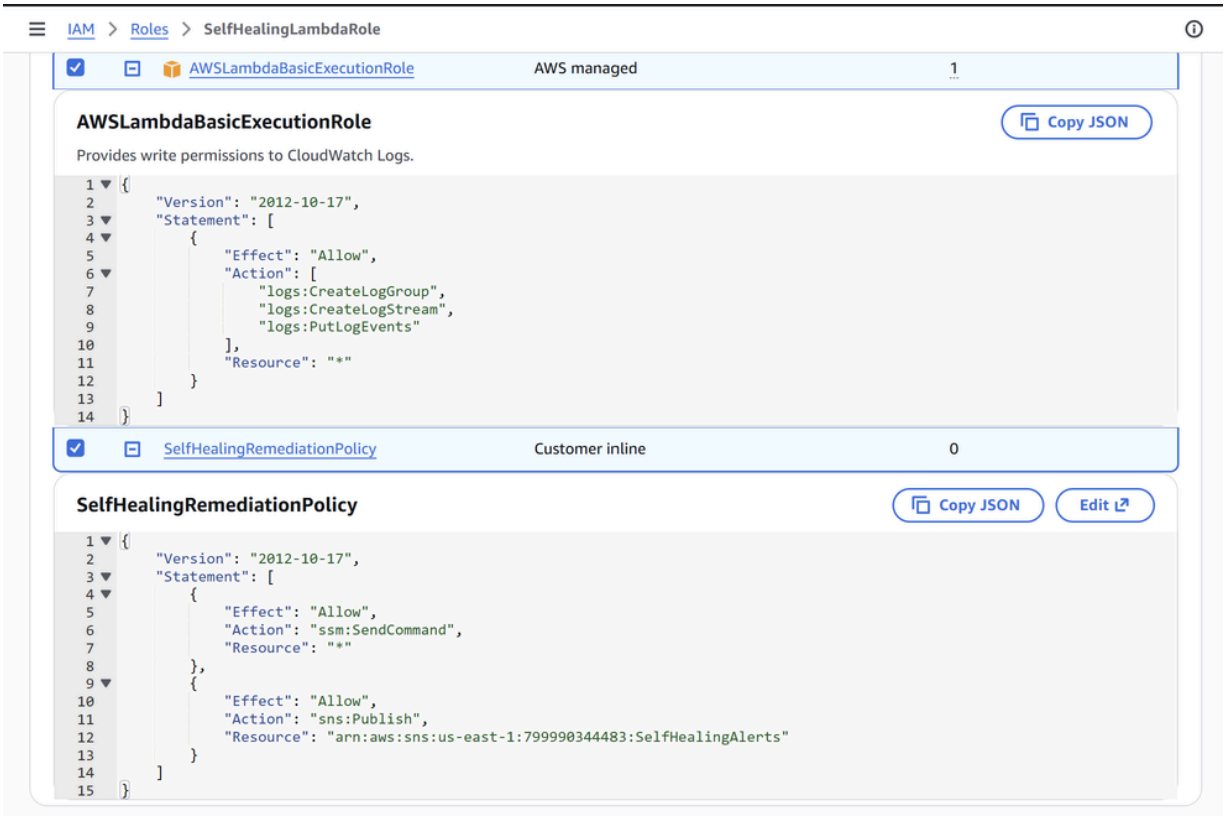


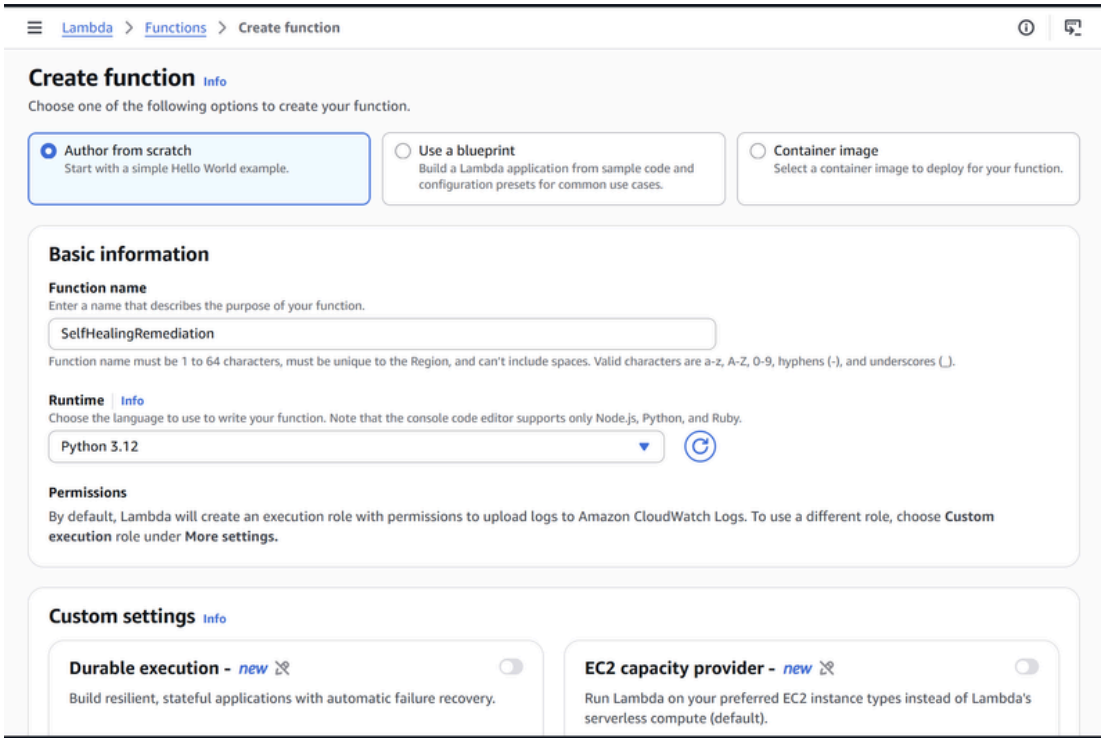


Lambda role inline policy

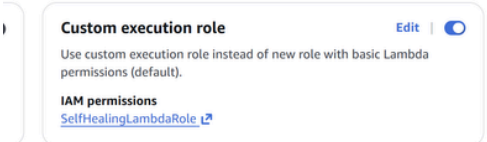


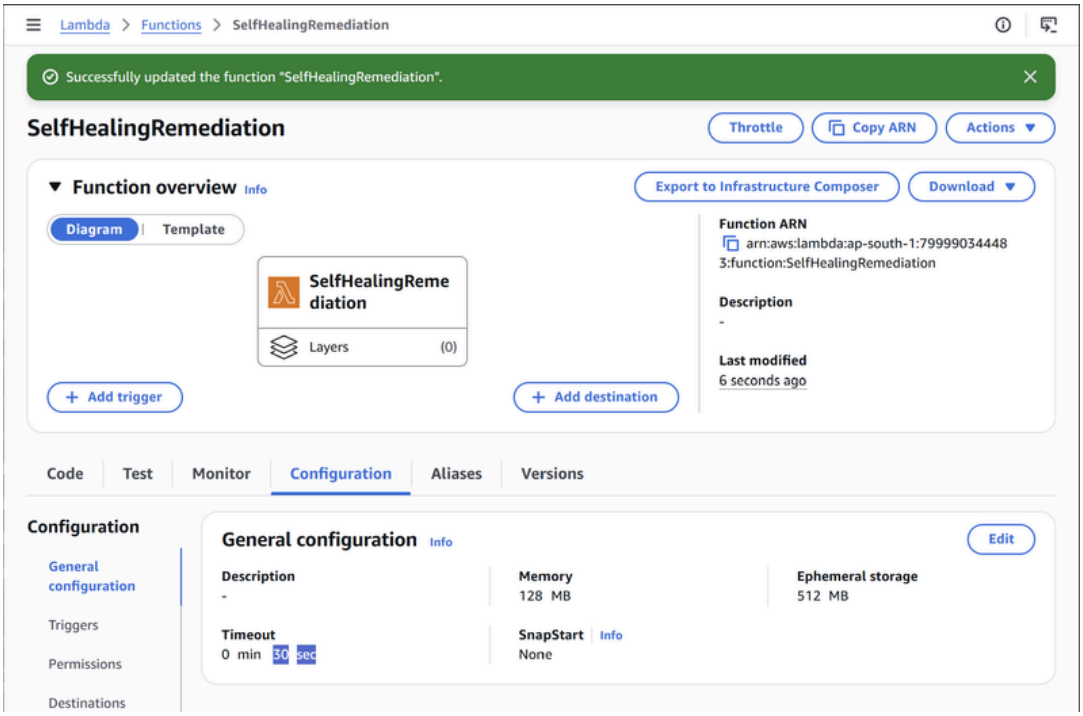
Lambda role with policy



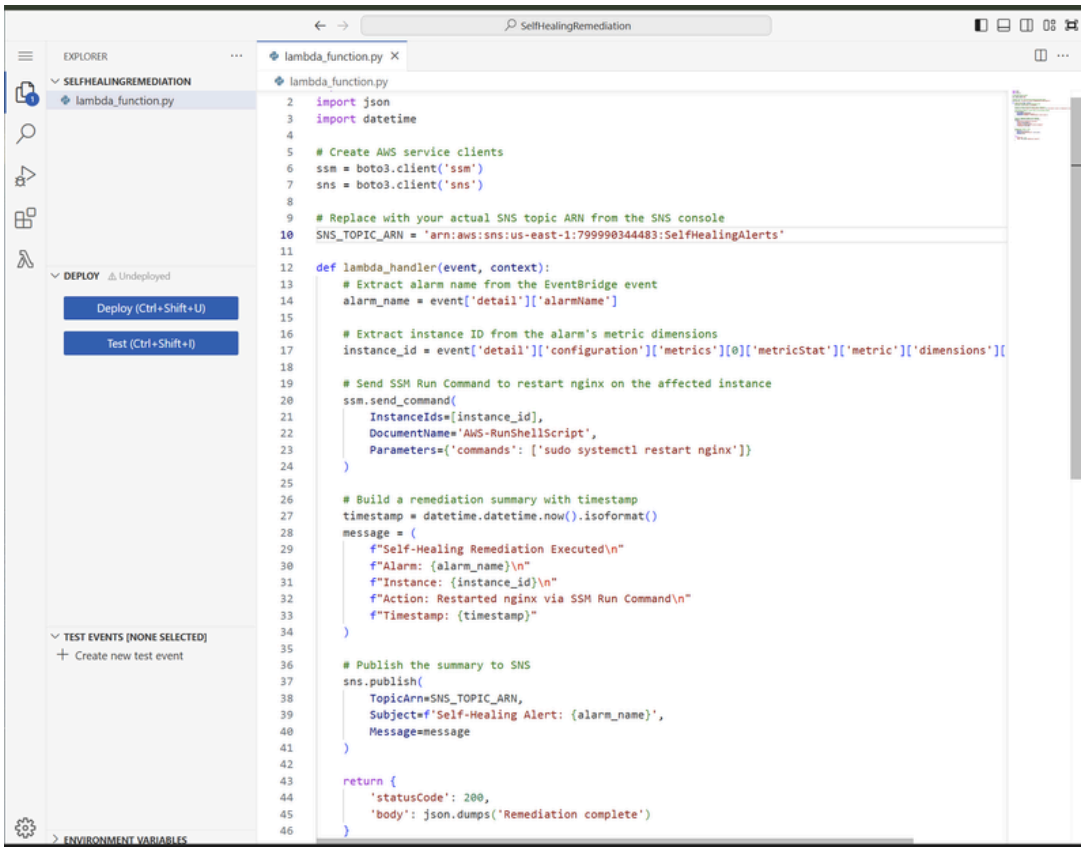


Lambda function





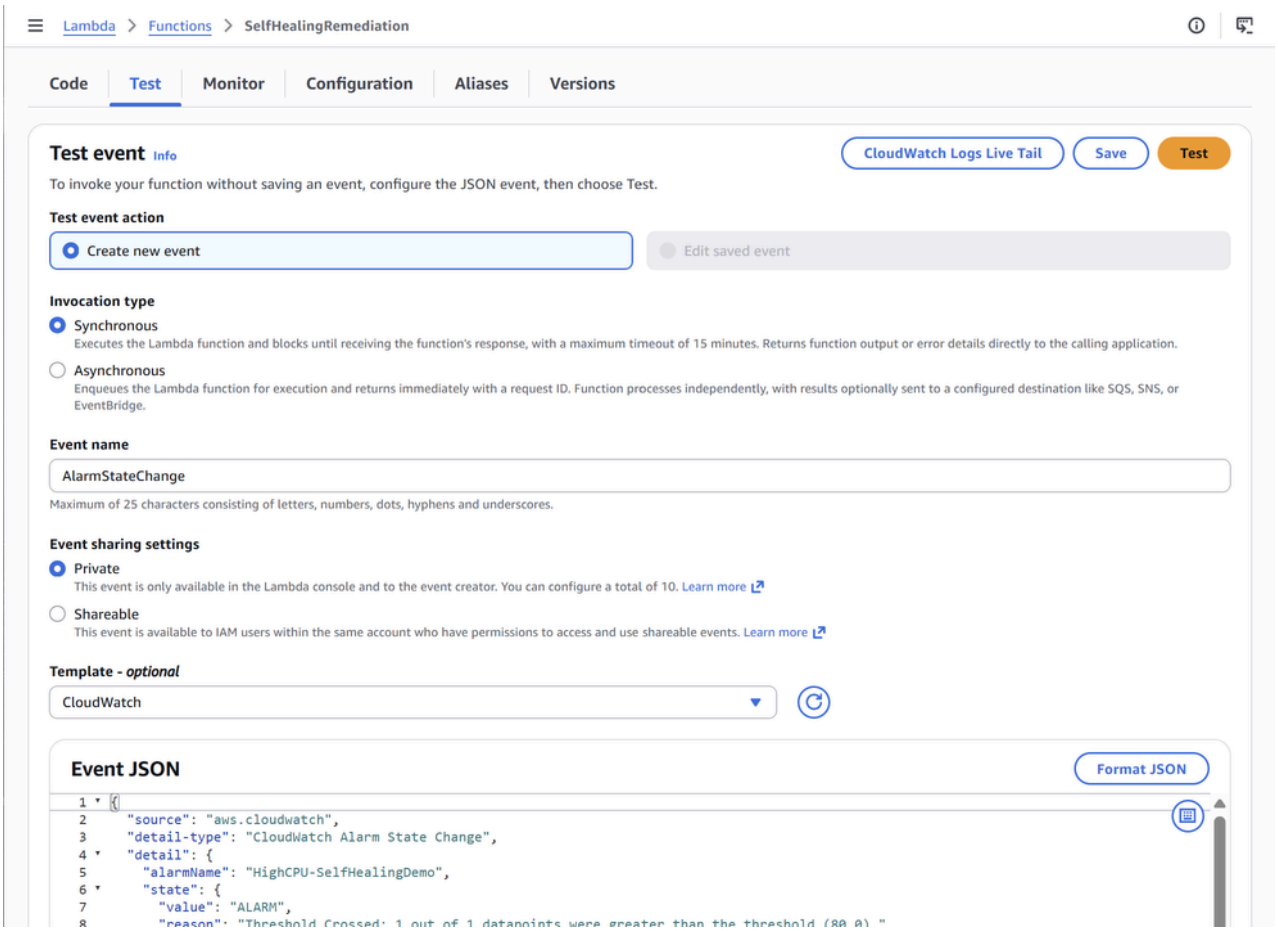
Lambda function configuration timeout



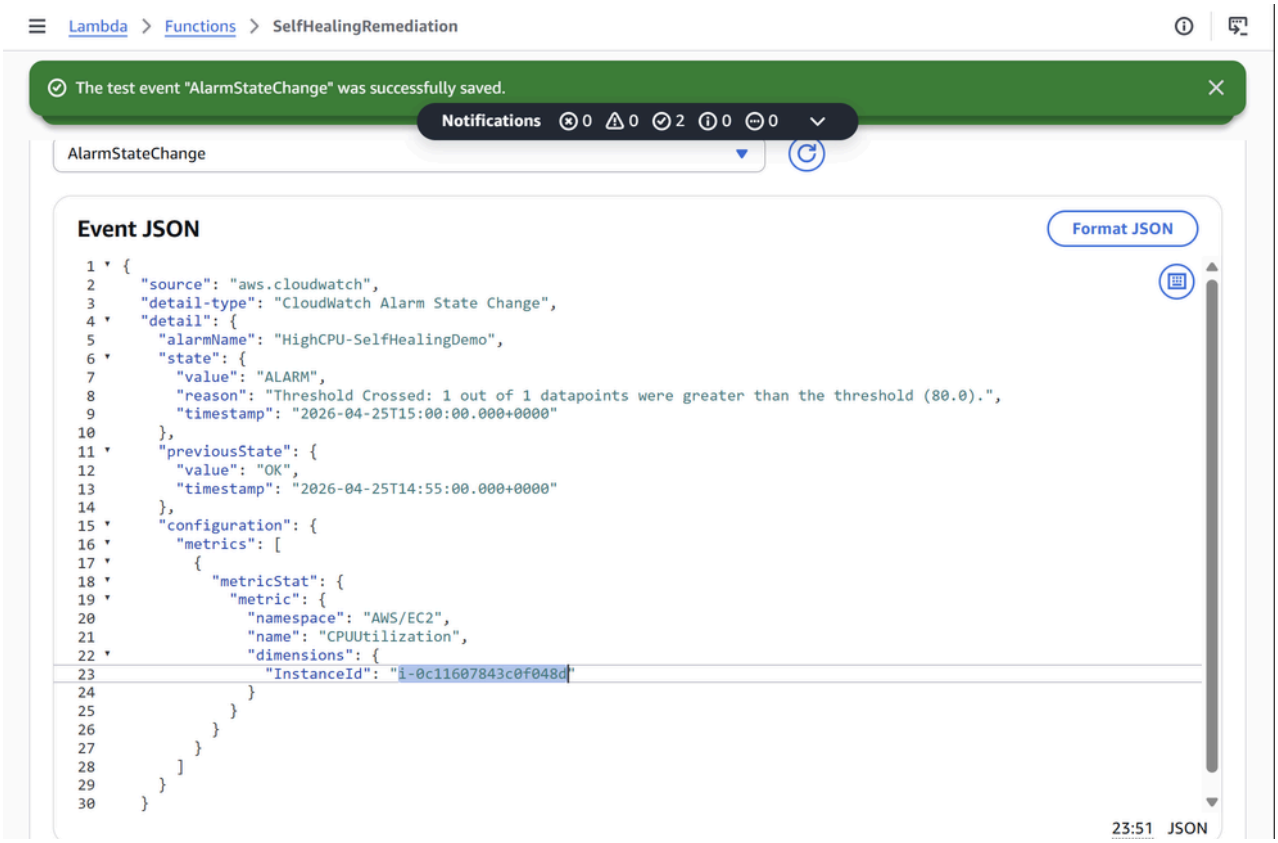
Lambda function code

How the Lambda function processes and responds to alarm events

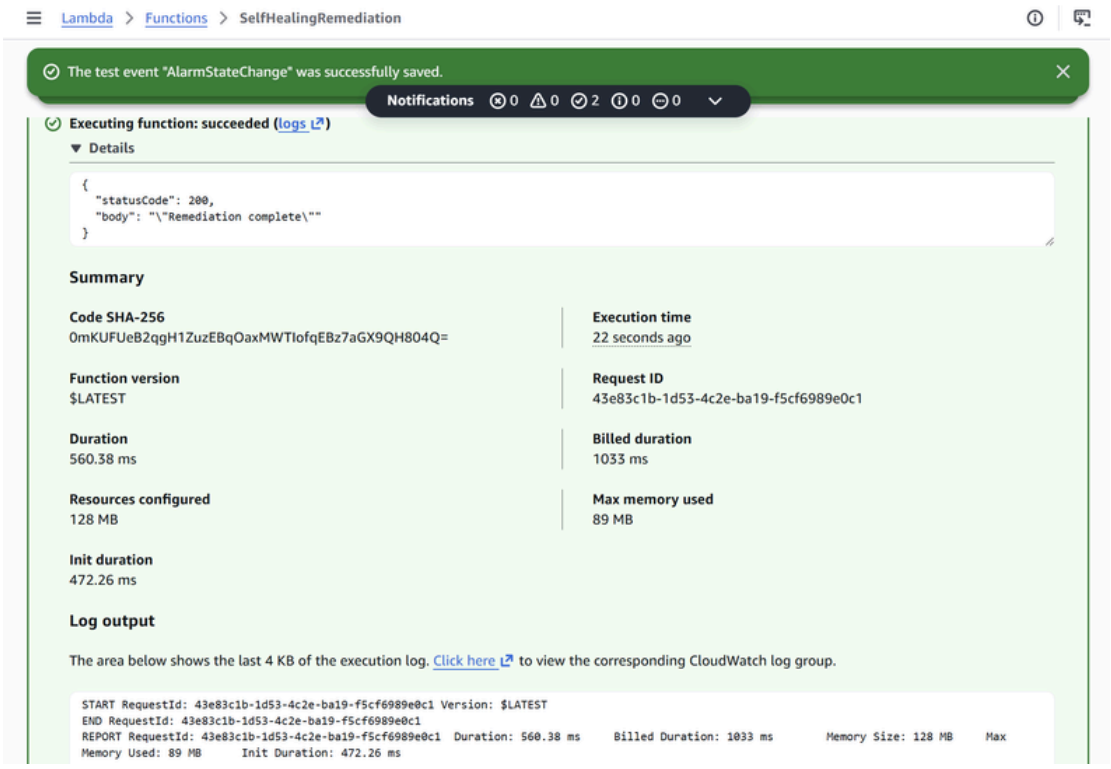
When my function receives an event, it first extracts the alarm name and EC2 instance ID from the EventBridge alarm payload. In this step, I use that instance ID to send an SSM Run Command that remotely restarts the nginx service on the affected EC2 instance. Then the function creates a remediation summary with the alarm name, instance ID, action taken, and timestamp. Finally, it publishes this message to the SNS topic so I receive an email notification confirming the system automatically healed the failure.

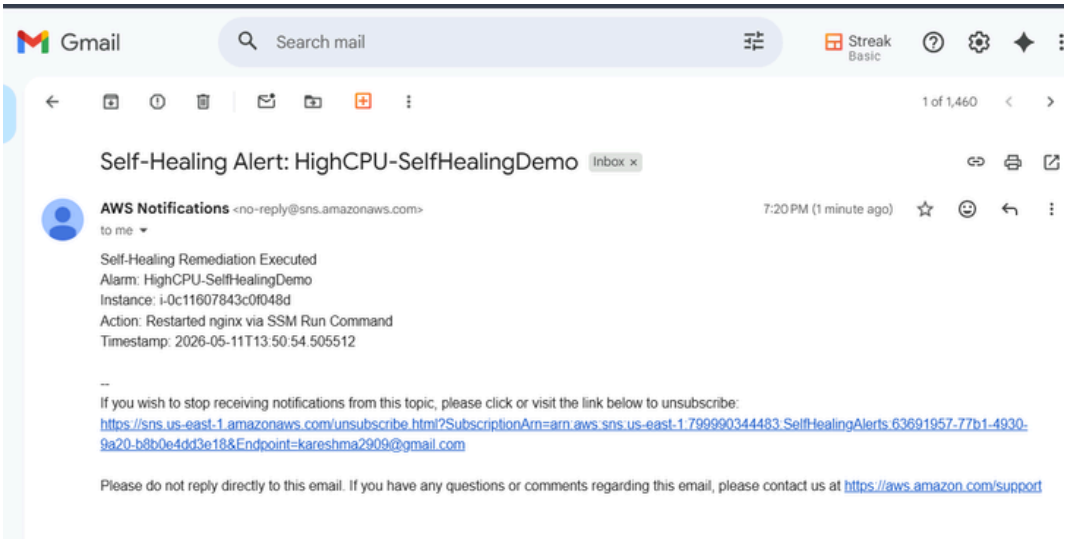


Lambda function test set up



Lambda function test JSON





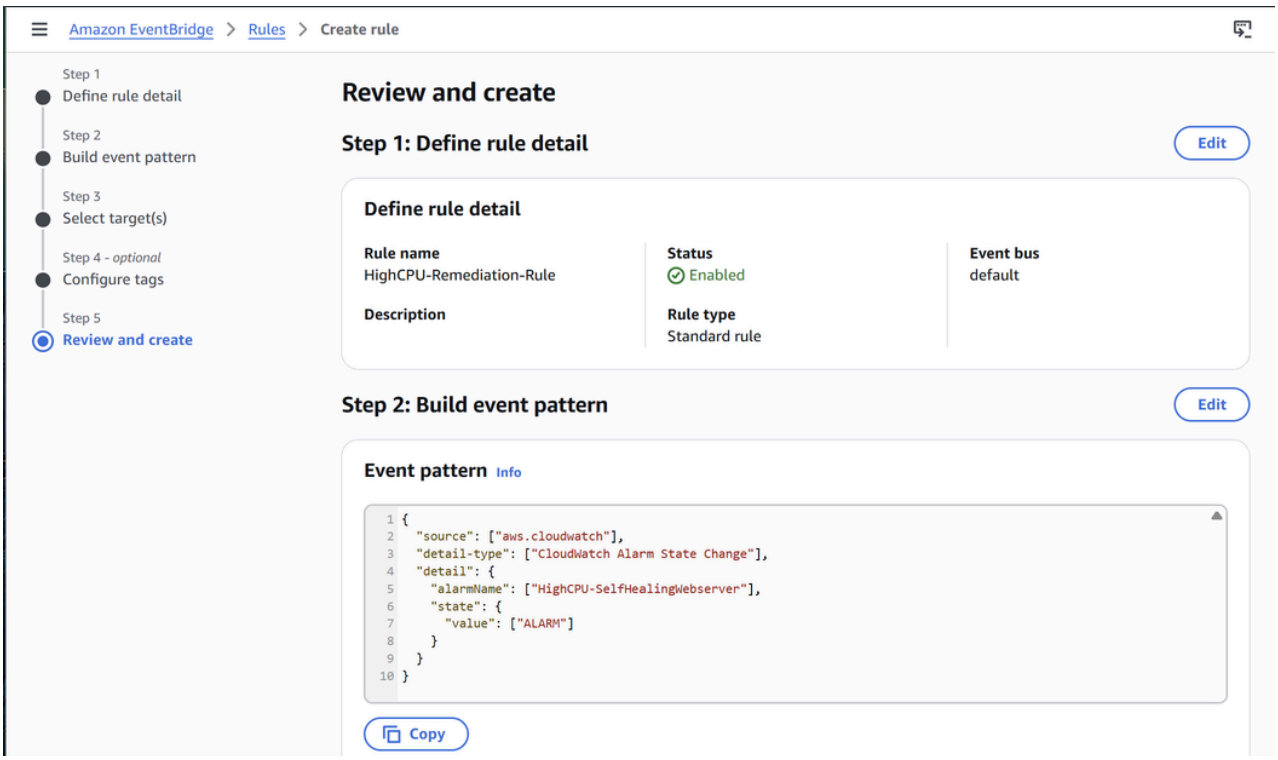
SNS notification to email inbox

My Lambda function was created in ap-south-1 (Mumbai) while EC2, SNS, and CloudWatch were in us-east-1 (N. Virginia), causing an InvalidInstanceId error. All AWS resources in a pipeline must be in the same region. Always check the region dropdown before creating resources.

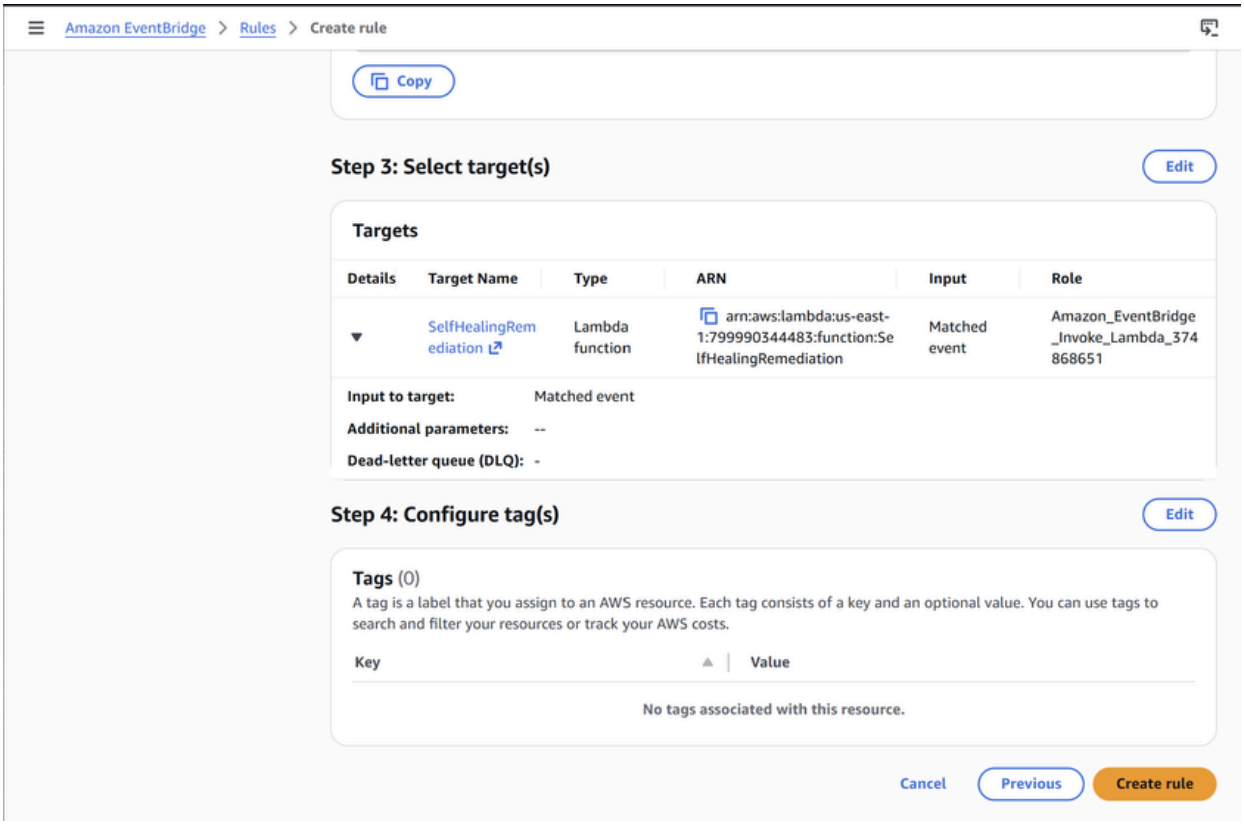
Wiring Up EventBridge to Route Events

Connecting the pipeline with event-driven routing

IIn this step, I'm creating an Amazon EventBridge rule that listens for my specific CloudWatch alarm event and routes it to my Lambda remediation function so that the system can automatically trigger the self-healing process whenever the alarm enters the ALARM state.

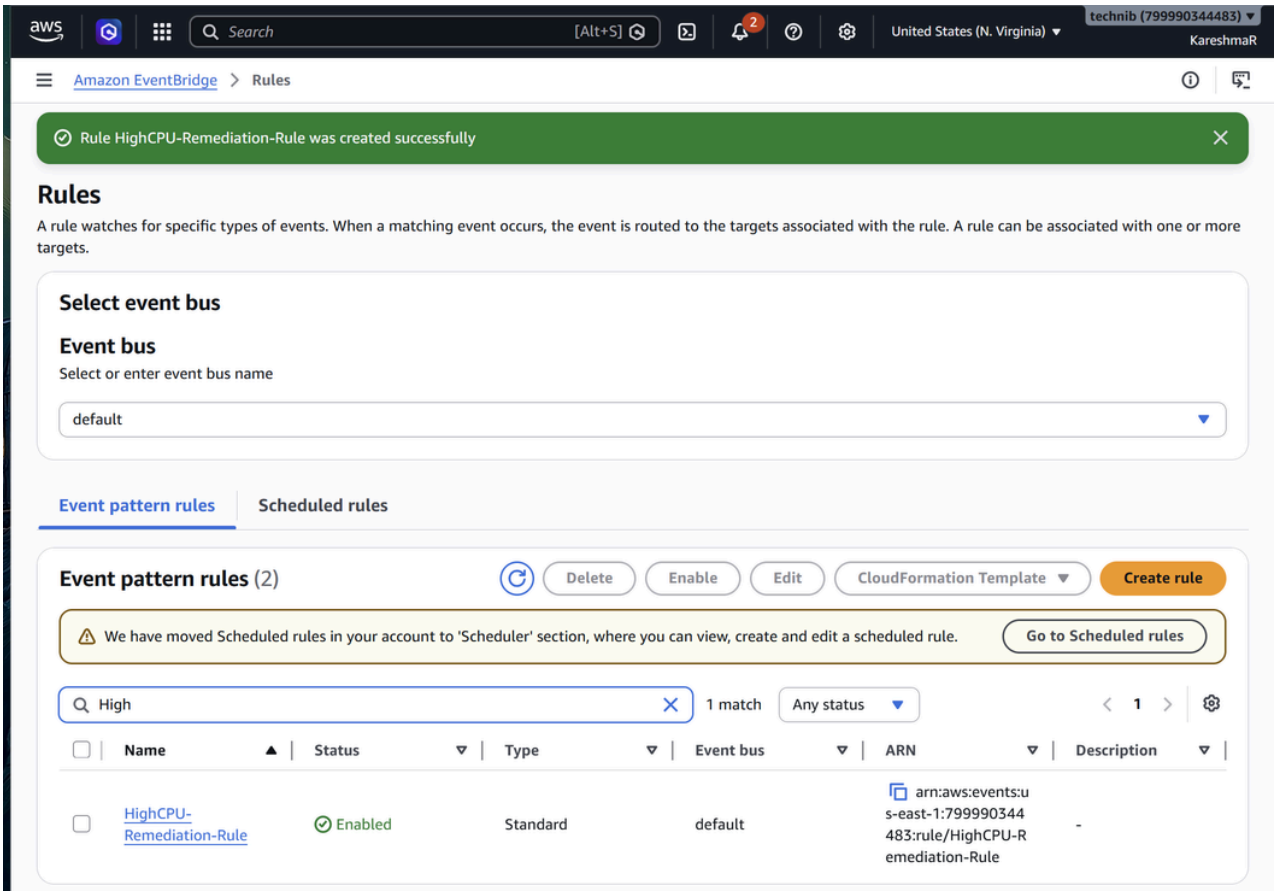


EventBridge rule



How the event pattern filters for the right triggers

The event pattern filters for events where the Amazon CloudWatch alarm state changes. It specifically checks that the event source is CloudWatch, the detail type is a CloudWatch Alarm State Change, and the alarm name matches HighCPU-SelfHealingWebserver. It also verifies that the alarm state value is ALARM. By defining these conditions, the rule ignores unrelated events and only captures the moment when this exact alarm enters the ALARM state. When this match happens, Amazon EventBridge triggers the AWS Lambda function to start the automated remediation process.



EventBridge rule enabled

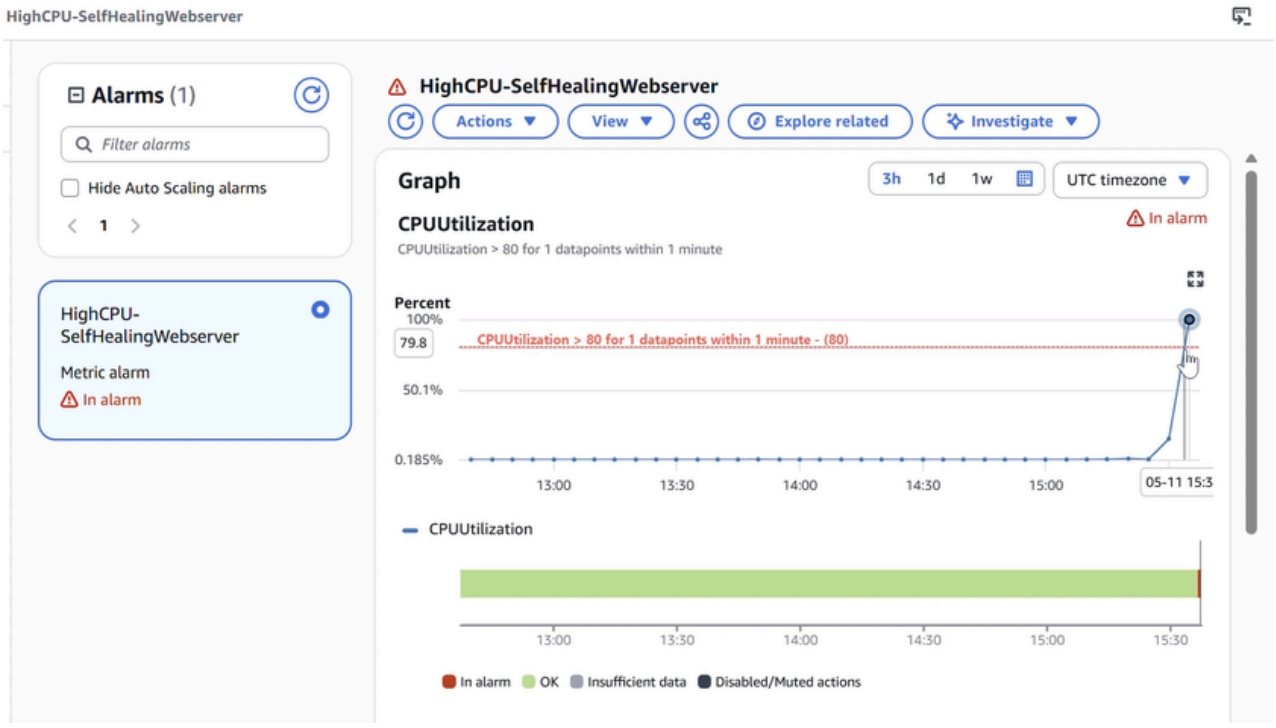
Proving the Self-Healing Pipeline Works End-to-End

Simulating failure and verifying automated recovery

In this step, I'm testing the entire self-healing pipeline by simulating a failure on my EC2 web server. I will intentionally stop nginx and create a CPU spike on the instance to trigger the Amazon CloudWatch alarm. This allows me to verify that the monitoring system detects the failure, Amazon EventBridge routes the alarm event, and the AWS Lambda function automatically restarts nginx and sends an alert through Amazon Simple Notification Service, confirming the self-healing system works end-to-end without manual intervention.

Testing the Self-Healing Pipeline

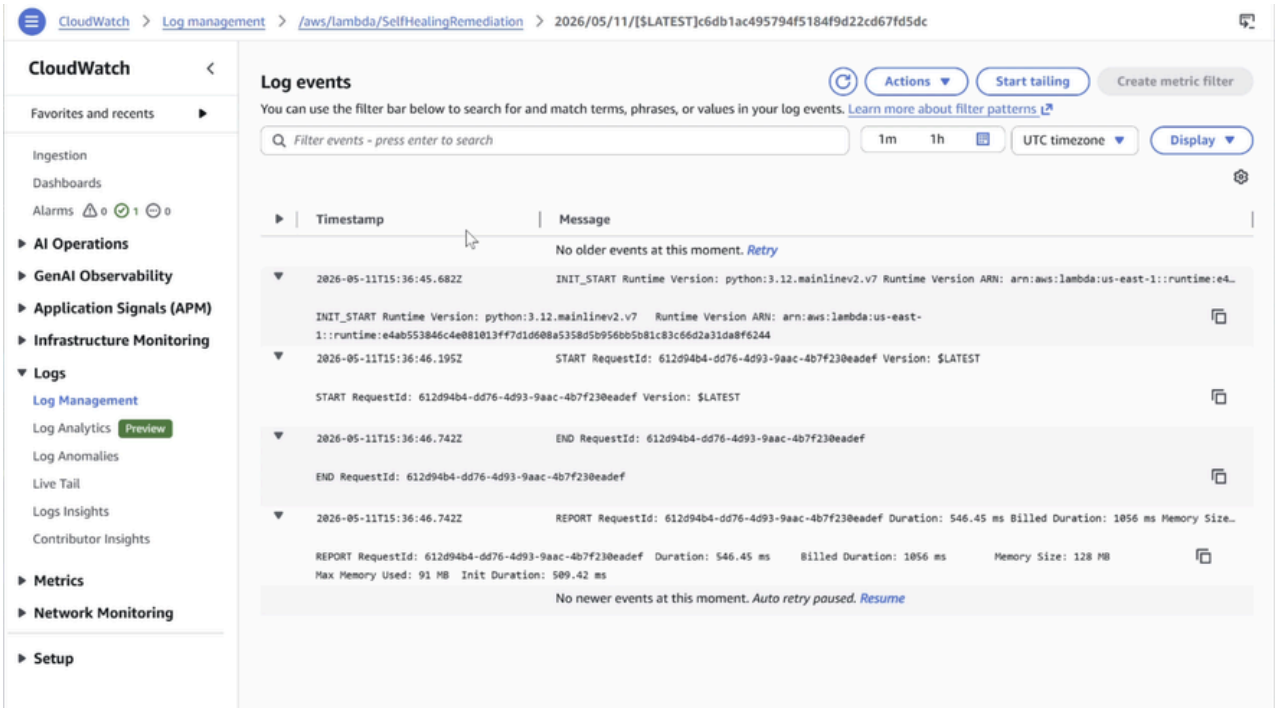
The pipeline was validated by intentionally stopping the nginx service and creating high CPU load using stress-ng. CloudWatch detected the anomaly, EventBridge triggered the Lambda function, and Systems Manager restarted nginx automatically. The system returned to a healthy state and a notification email confirmed the remediation.



CloiudWatch In alarm after CPU spike

The full remediation chain from alarm to restored nginx

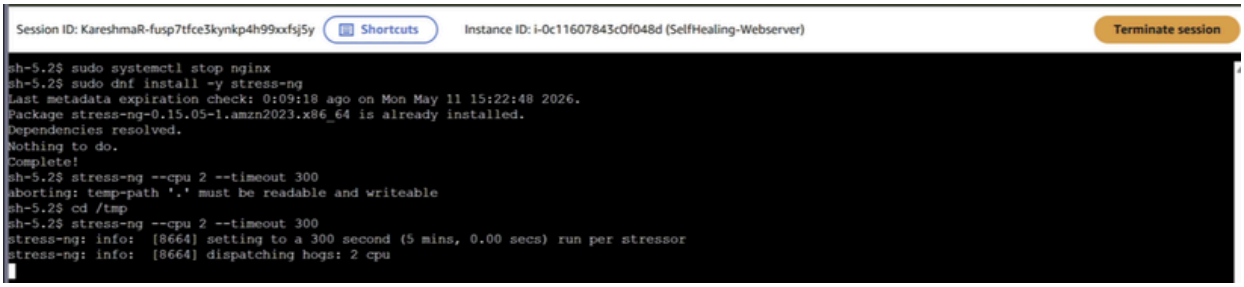
After the alarm fired, Amazon CloudWatch generated an alarm state change event when CPU utilization exceeded the configured threshold. Amazon EventBridge captured this event using the rule I created and routed it to my AWS Lambda remediation function. The Lambda function extracted the alarm and instance details from the event and used AWS Systems Manager to send a Run Command that restarted the nginx service on the EC2 instance. Once nginx was restarted successfully, the function published a message through Amazon Simple Notification Service, confirming the automated remediation and restoring the web server without manual intervention.

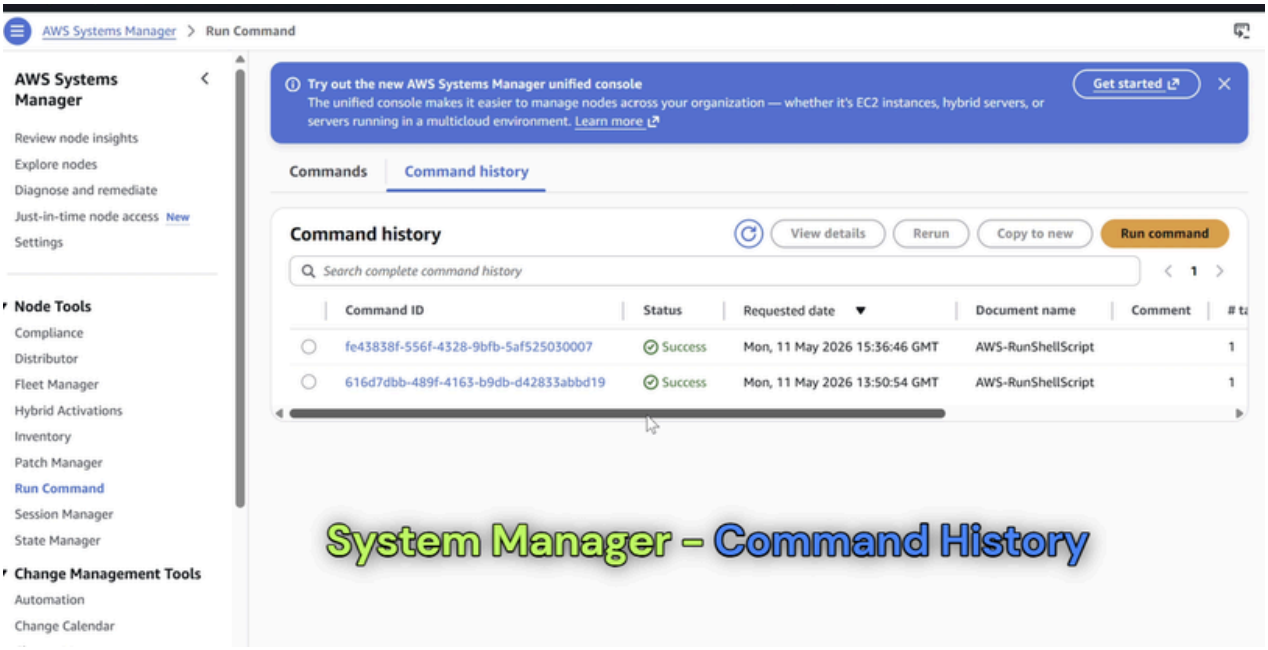


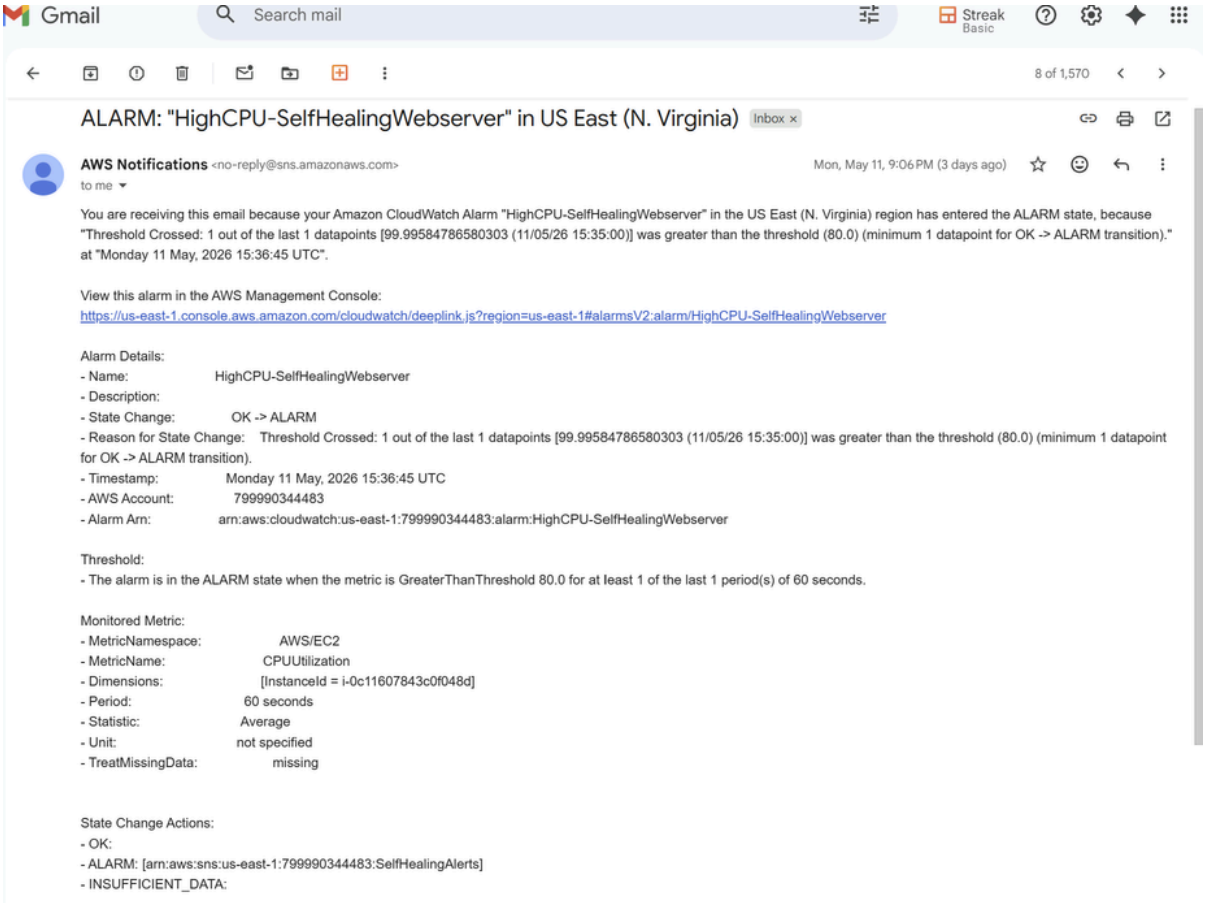
Log stream and events

Simulate a web server failure by stopping nginx and spiking CPU

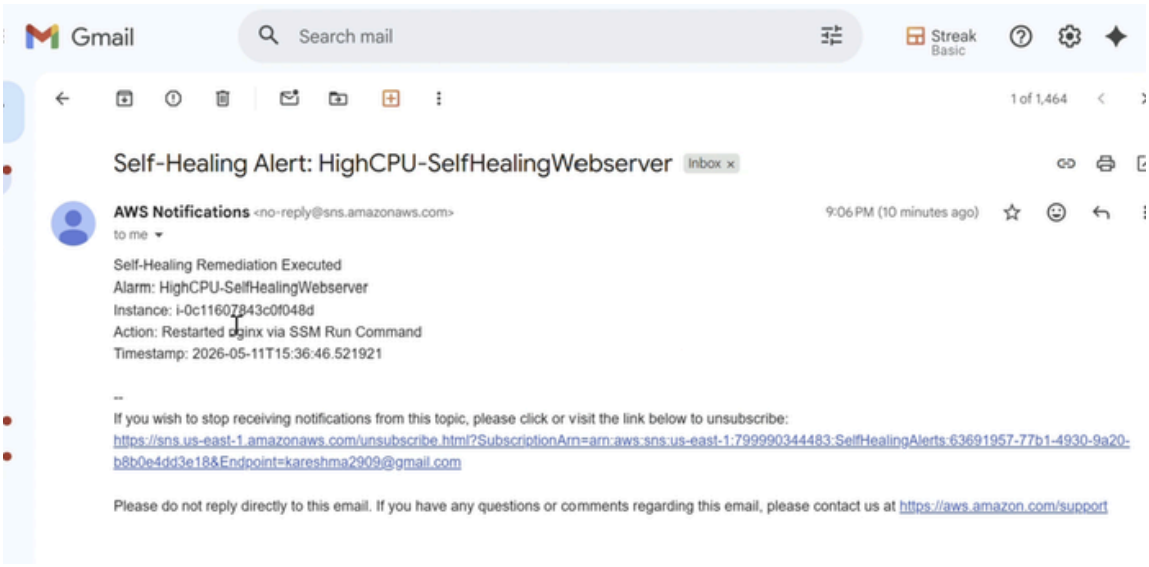
stress-ng failed with "temp-path '.' must be readable and writeable" in Session Manager because the current directory lacked write access. Running "cd /tmp" before executing stress-ng resolved it, as /tmp is always readable and writeable on Linux instances.





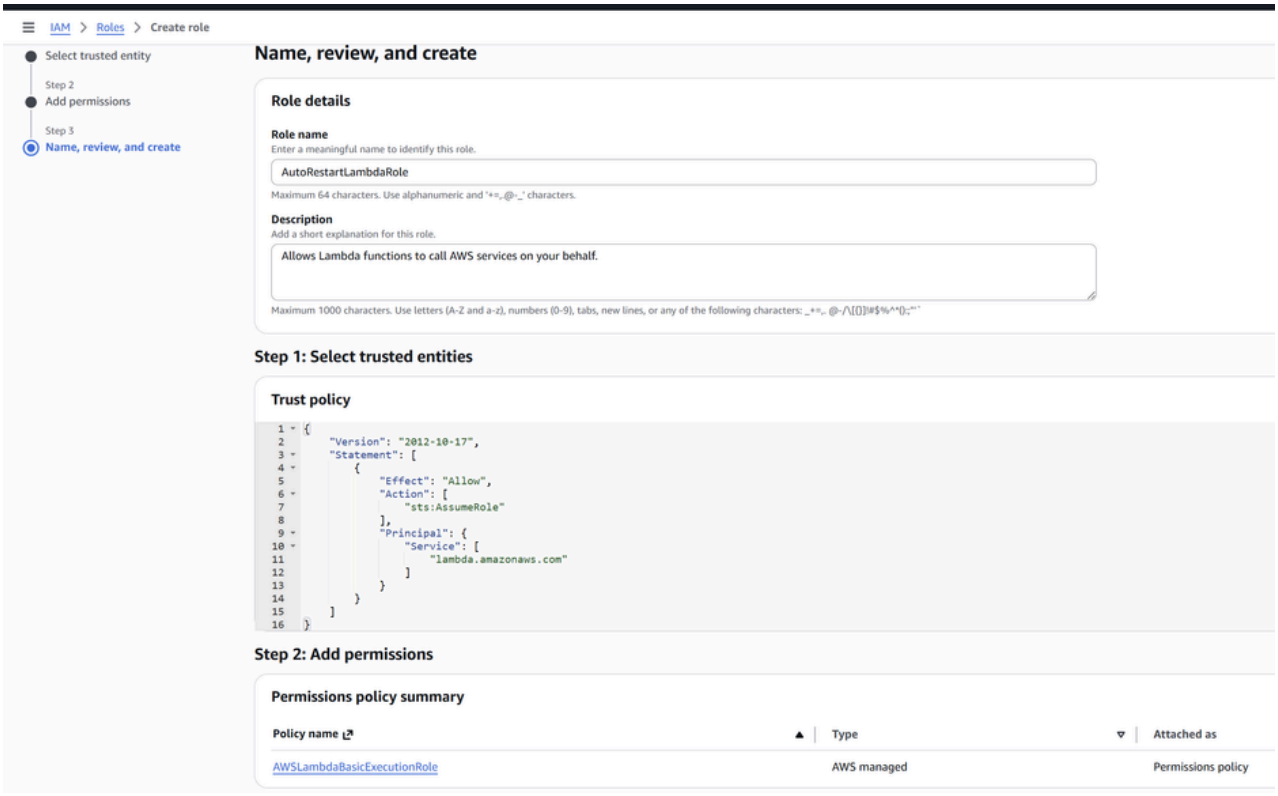

Gmail
Search mail
Streak Basic
8 of 1,570
ALARM: "HighCPU-SelfHealingWebserver" in US East (N. Virginia)
Inbox ×
AWS Notifications <no-reply@sns.amazonaws.com>
to me
Mon, May 11, 9:06 PM (3 days ago)
You are receiving this email because your Amazon CloudWatch Alarm "HighCPU-SelfHealingWebserver" in the US East (N. Virginia) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [99.99584786580303 (11/05/26 15:35:00)] was greater than the threshold (80.0) (minimum 1 datapoint for OK -> ALARM transition)." at "Monday 11 May, 2026 15:36:45 UTC".
View this alarm in the AWS Management Console:
https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/HighCPU-SelfHealingWebserver
Alarm Details:
- Name: HighCPU-SelfHealingWebserver
- Description:
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [99.99584786580303 (11/05/26 15:35:00)] was greater than the threshold (80.0) (minimum 1 datapoint for OK -> ALARM transition).
- Timestamp: Monday 11 May, 2026 15:36:45 UTC
- AWS Account: 799990344483
- Alarm Arn: arn:aws:cloudwatch:us-east-1:799990344483:alarm:HighCPU-SelfHealingWebserver
Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanThreshold 80.0 for at least 1 of the last 1 period(s) of 60 seconds.
Monitored Metric:
- MetricNamespace: AWS/EC2
- MetricName: CPUUtilization
- Dimensions: [InstanceId = i-0c11607843c0f048d]
- Period: 60 seconds
- Statistic: Average
- Unit: not specified
- TreatMissingData: missing
State Change Actions:
- OK:
- ALARM: [arn:aws:sns:us-east-1:799990344483:SelfHealingAlerts]
- INSUFFICIENT_DATA:

SNS Email


Gmail
Search mail
Streak Basic
1 of 1,464
Self-Healing Alert: HighCPU-SelfHealingWebserver
Inbox ×
AWS Notifications <no-reply@sns.amazonaws.com>
to me
9:06 PM (10 minutes ago)
Self-Healing Remediation Executed
Alarm: HighCPU-SelfHealingWebserver
Instance: i-0c11607843c0f048d
Action: Restarted nginx via SSM Run Command
Timestamp: 2026-05-11T15:36:46.521921
--
If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
https://sns.us-east-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:us-east-1:799990344483:SelfHealingAlerts:63691957-77b1-4930-9a20-b8b0e4dd3e18&Endpoint=kareshma2909@gmail.com
Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at https://aws.amazon.com/support

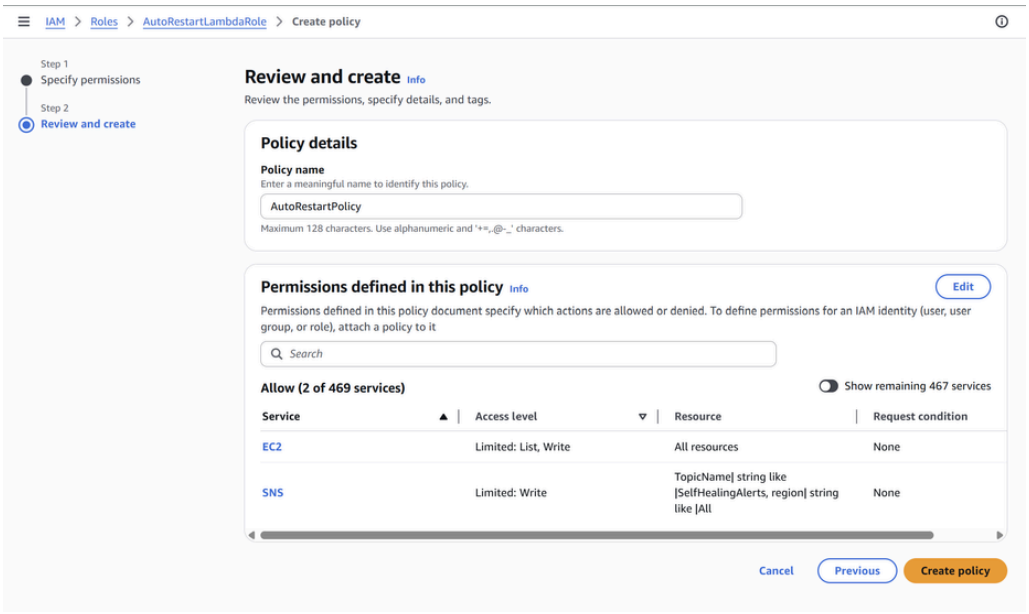
Extra: Auto-Restarting Stopped EC2 Instances

A second automation was implemented to restart EC2 instances that were stopped accidentally. A dedicated Lambda function checks whether an instance contains an AutoRestart tag before restarting it. This ensures only opted-in instances are automatically recovered.

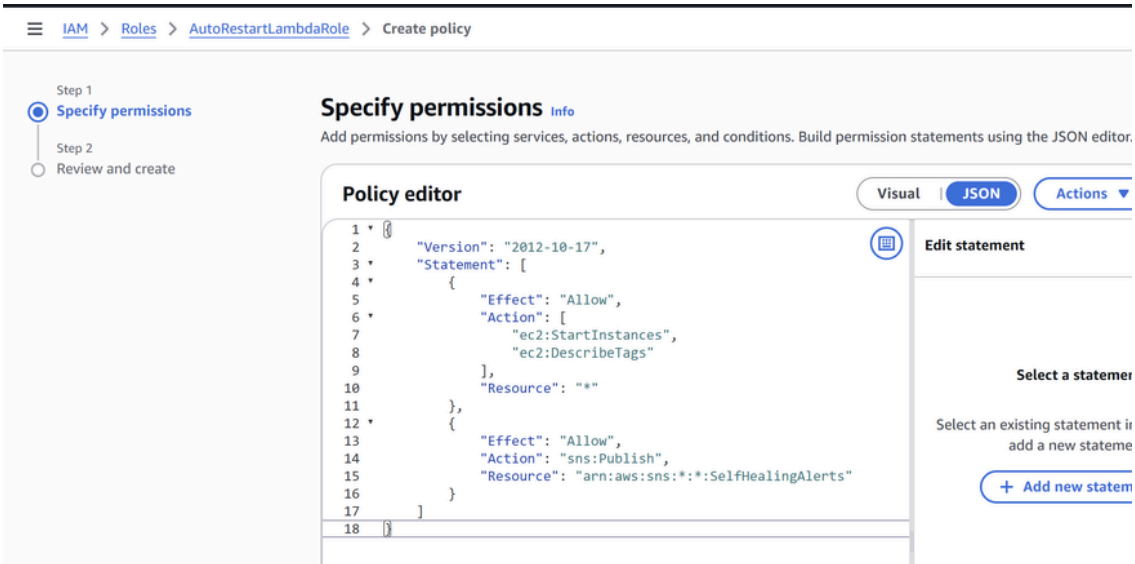


IAM Permission to Auto Restart EC2 when stopped

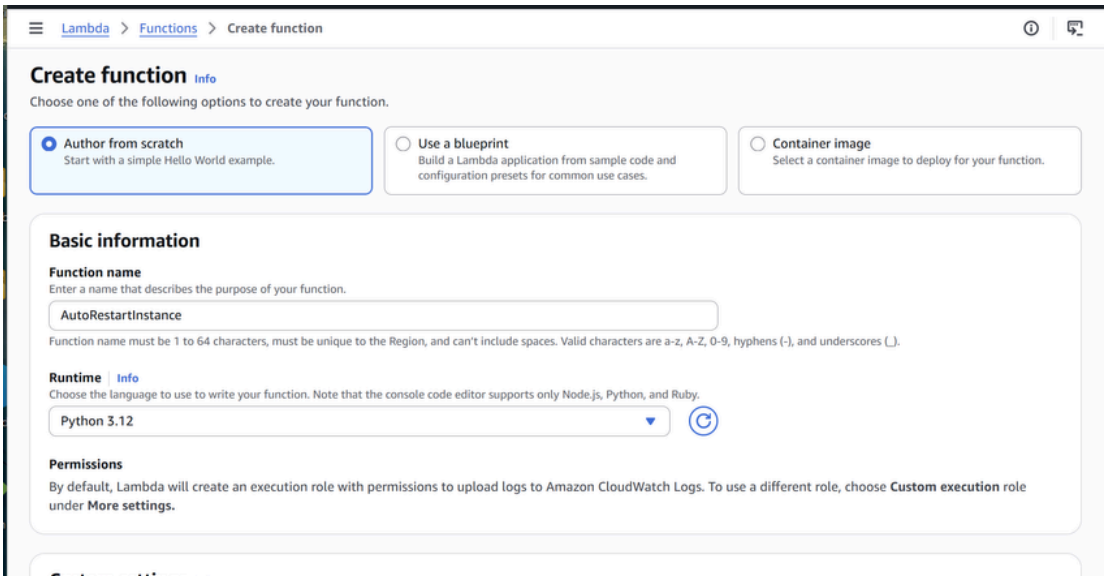
Autorestart policy attach to AutorestratLambdaRole

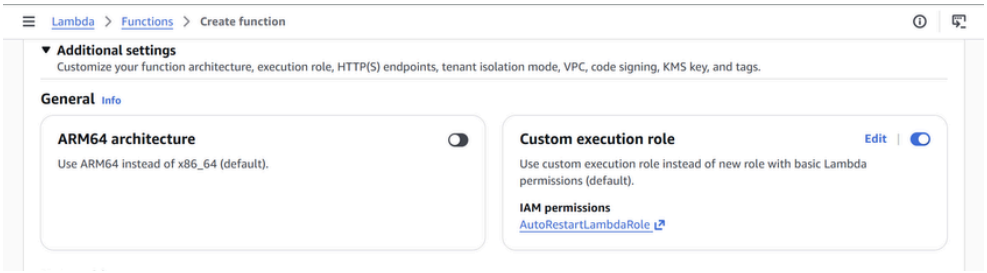


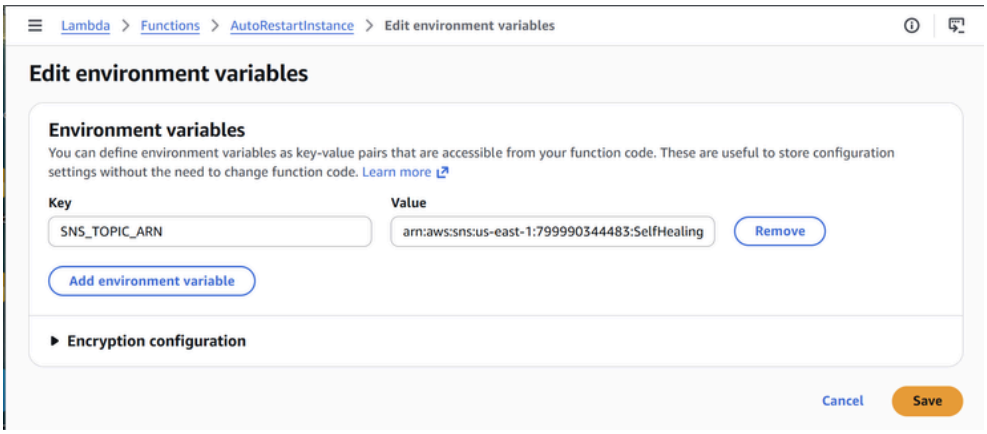
IAM Permission policy to Auto Restart EC2 when stopped

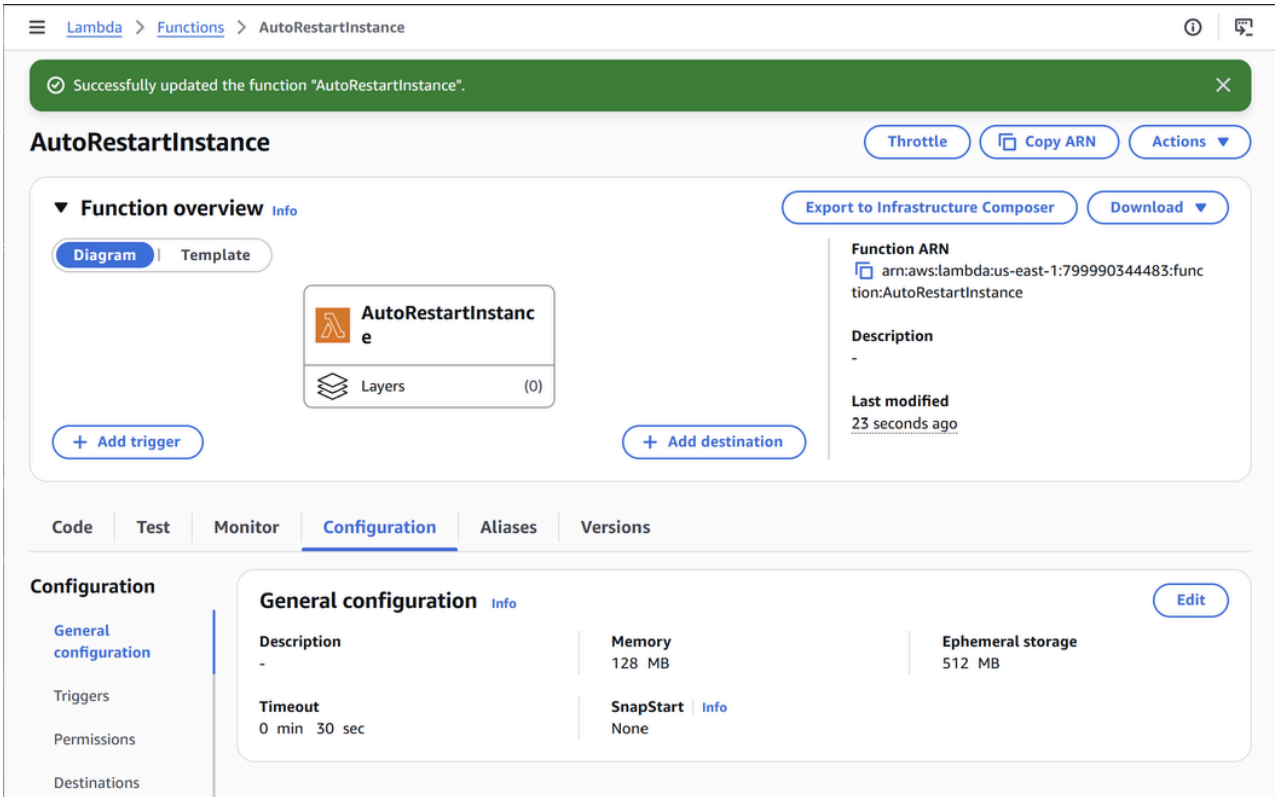


AutoRestart Lambda function and code

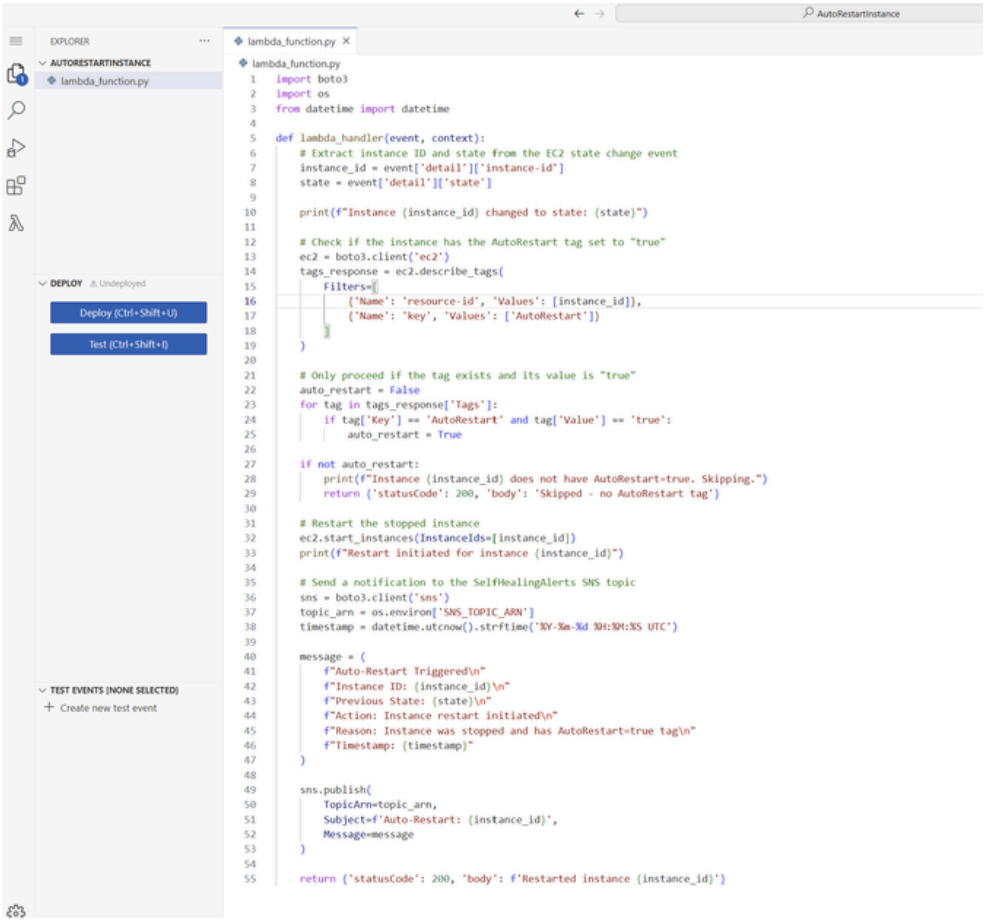


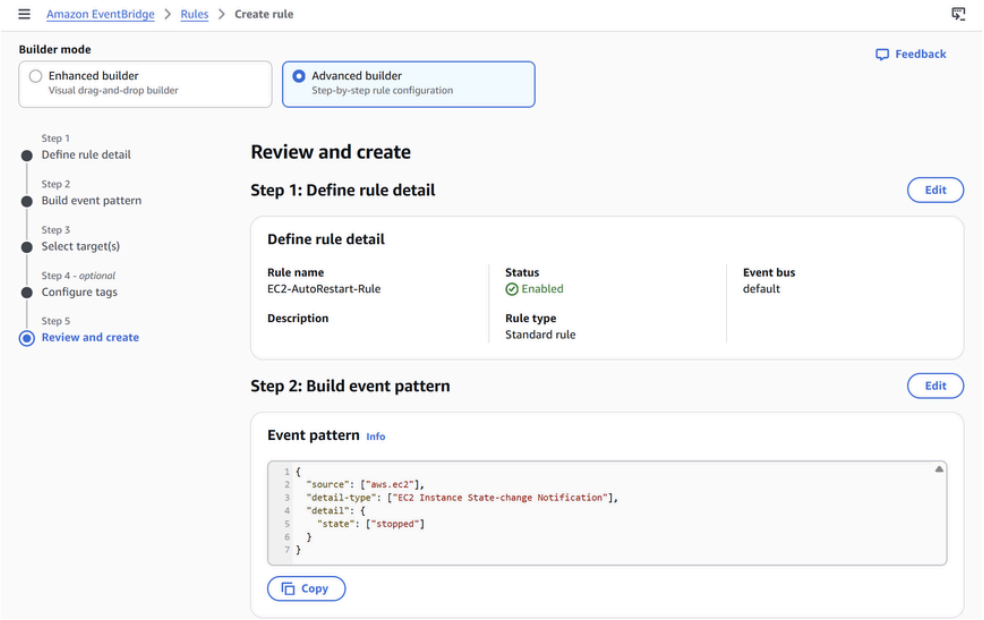




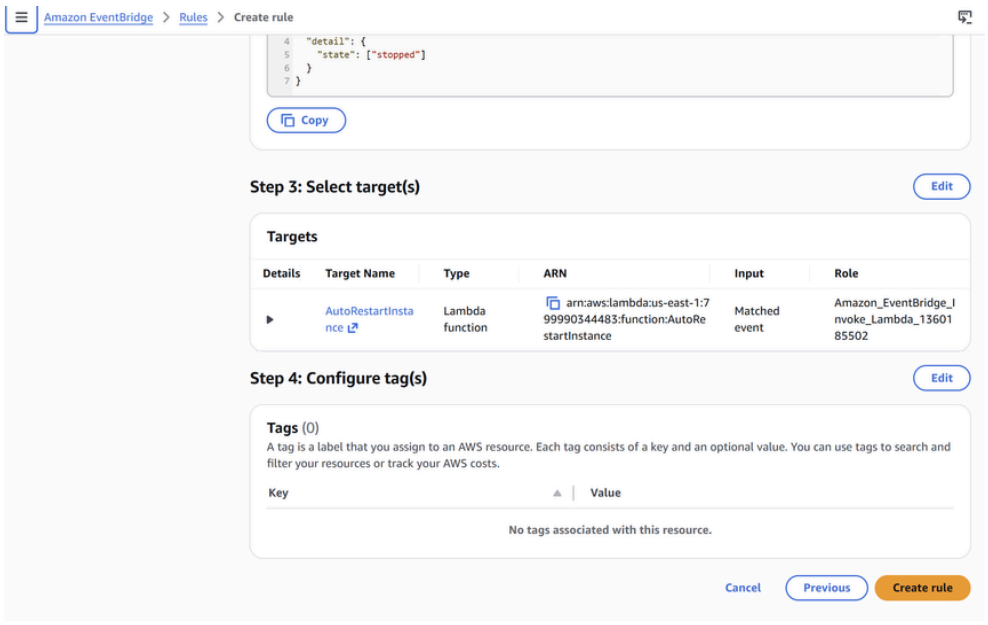


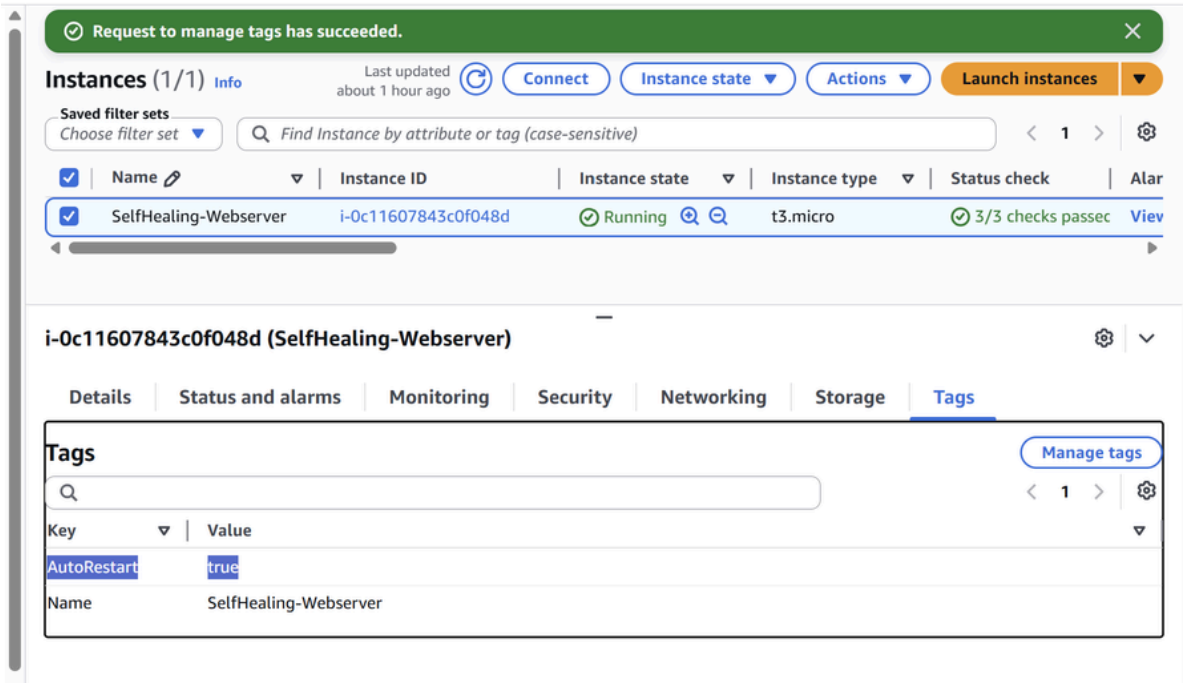
Lambda function configuration





EventBridge rule stopped event



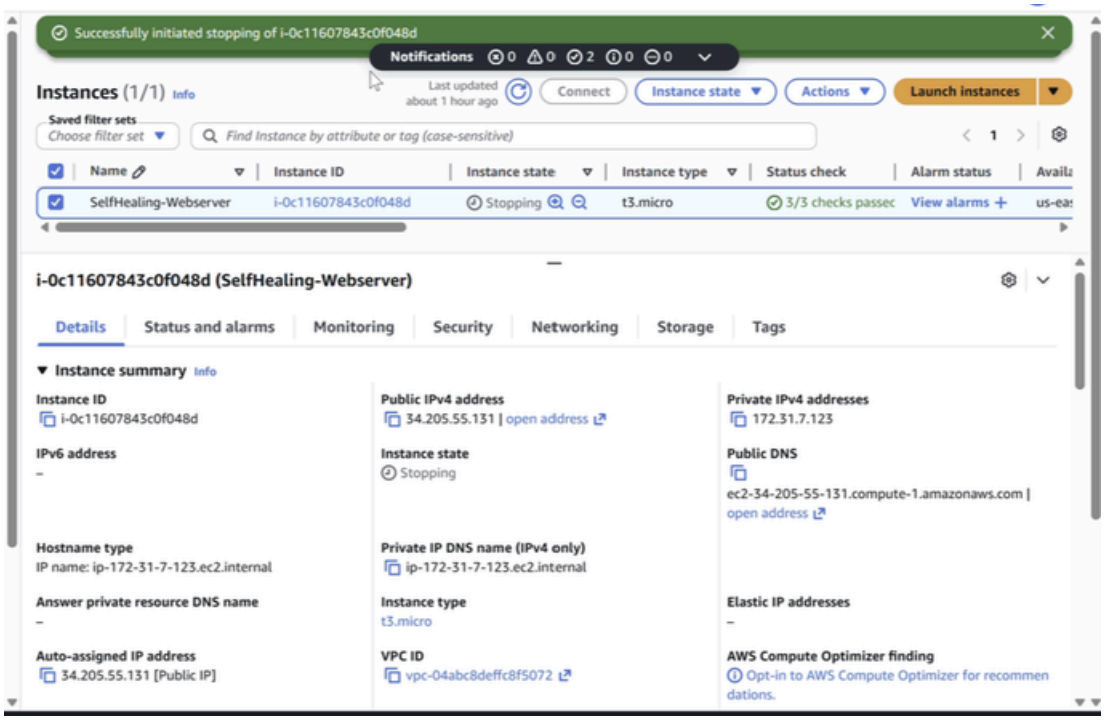


Tag Instance

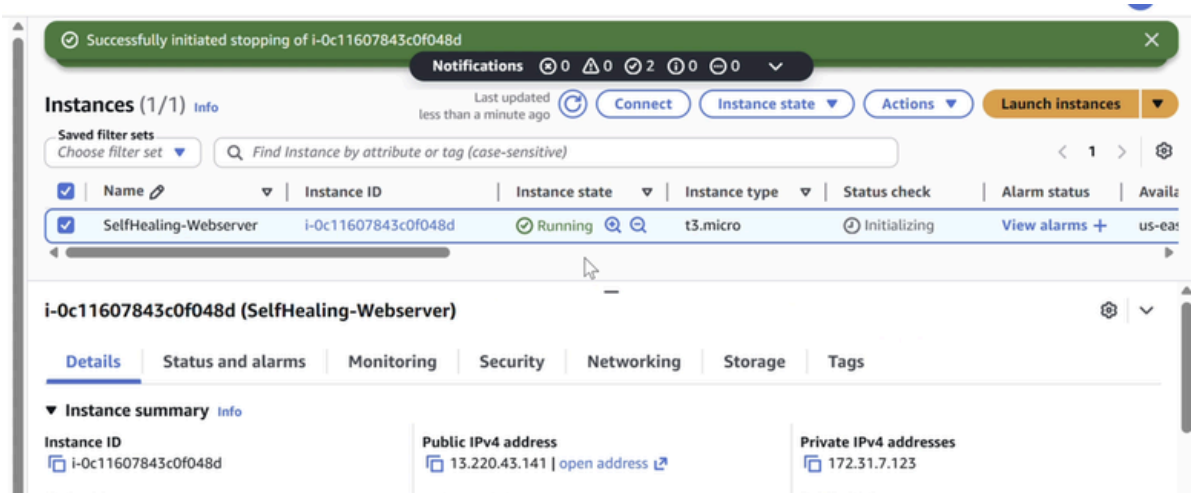
Tag-based filtering for safe production automation

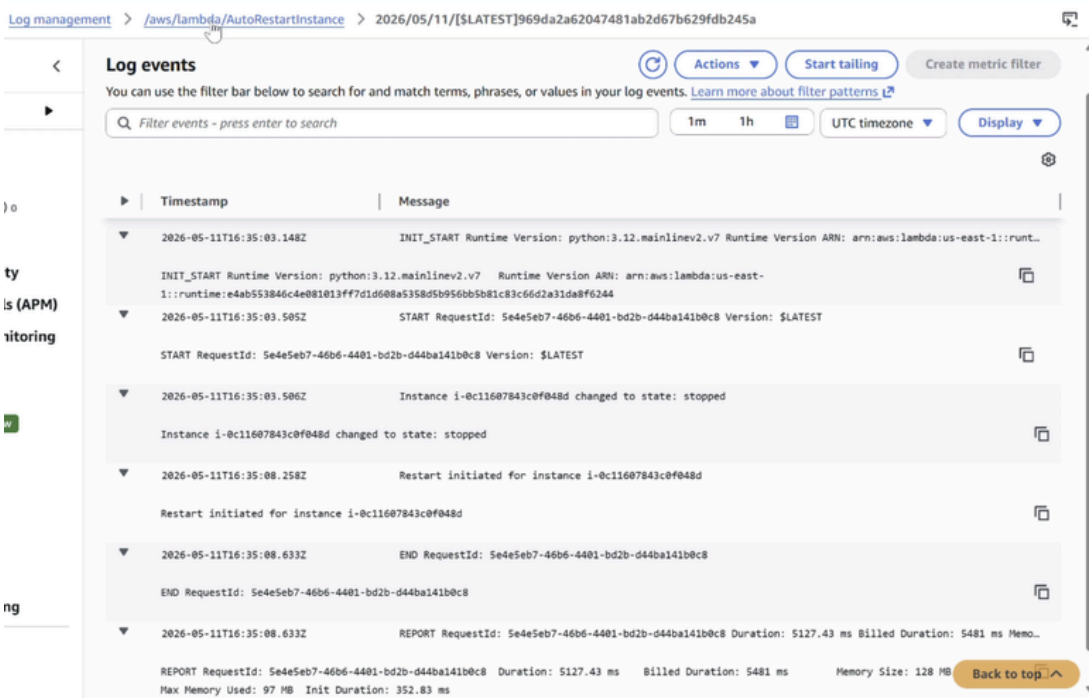
In this project extension, I added a tag check that uses Amazon EC2 instance tags to determine whether an instance should be restarted automatically. The AWS Lambda function receives the stop event, then calls the EC2 API to read the instance tags and look for AutoRestart=true. If the tag exists, the function starts the instance; if not, it safely skips the action.

This filter is important in production because not every stopped instance should be restarted. Some are intentionally stopped for maintenance, cost control, or decommissioning. The tag creates an explicit opt-in mechanism so only selected instances are automatically restarted.

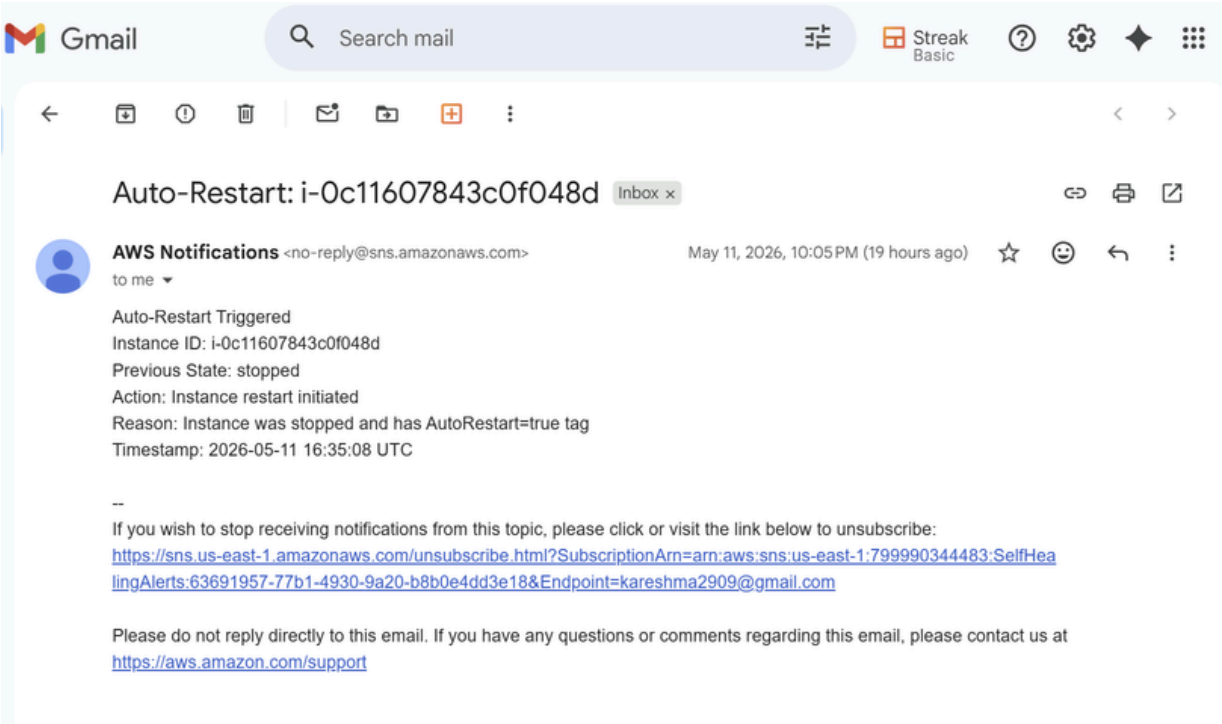


Instance stopped to fire event rule





Log groups



Email notification for Autorestart Instance

Key Learnings

- Designing event driven automation pipelines
- Using CloudWatch alarms for operational monitoring
- Automataing remedaiation using AWS Lambda
- Using Systems Manager for remote command execution
- Implementing guardrails with tag based automation

Future Improvements

- Extend the system with AI-driven incident analysis using Amazon Bedrock
- Observability dashboards
- DevOps Copilot interface for querying infrastructure health
- Additional health checks for nginx
- Auto Scaling integration
- Implement automated rollback strategies
- Add centralized logging dashboards
- Integrate Slack or incident management tools for team alerts

Cost Considerations

- The architecture is designed around AWS Free Tier services.
- EC2 t3.micro: Free tier eligible
- Lambda: Always free tier usage
- CloudWatch alarms: Free tier limits
- EventBridge: Free tier event volume
- SNS notifications: Minimal cost

Cleanup

To avoid charges after experimentation:

1. Terminate EC2 instance
2. Delete Lambda functions
3. Remove EventBridge rules
4. Delete SNS topic and subscription
5. Delete CloudWatch alarm
6. Remove IAM roles
7. Delete security group allowing HTTP on port 80.

Kareshma
Rajaananthapadmanaban

Follow up for more interesting projects !!!

Check out my profile for more projects